AI Document-Extraction Service

Phase 1 — the v2 design explained

Technical review of the Phase 1 plan by Claude and Codex, and the corrected design

53	14	32	7
findings after merging both reviews	High — fixed in the design	Medium	Low

> Unchanged and verified: the HMAC test vector (signature recomputed exactly), the worked cost examples, the coverage arithmetic, the managed-identity design, the transactional outbox and the tenant isolation (404 for another consumer's job).

> Three decisions taken by the owner: (1) target **.NET 10 LTS** — .NET 9 support ends on 10 November 2026; (2) the pilot upload action gets **[Authorize]** and the kit refuses anonymous calls; (3) the revised plan v2 will be written after this explanation is approved.

Source plan: `AI-Extraction-Service-Plan.md`, revision of 23 October 2026 (Phase 1 only).
Pilot consumer checked: `ForwardingUniFreight.Solution / Forwarding.MvcApp`.
Reviewers: Claude (independent review, repository and documentation checks, arithmetic) and Codex `gpt-6-astra` (read-only review, 35 findings, one debate round).
Date: 24 October 2026. Status: design explanation — nothing has been implemented.

Contents

1. Summary — what changes and why
2. How the review was done
3. Scope (unchanged)
4. Solution structure
5. Where each part lives
6. Runtime view and trust boundaries
7. Request cycle — submit
8. Request cycle — the worker
9. Request cycle — deliver and store
10. The worker in detail and the retry rules
11. State machines
12. The two concurrency fixes: lease token and run number
13. Database — DocAi
14. Database — company side
15. Database permissions
16. Cost per application — model and formula
17. Cost calculations — worked examples
18. Month end, reconciliation gate and capacity cost
19. Security and identity changes
20. Exceptions, error codes and alerts
21. The MVC 4.8 kit
22. Coverage of the five concerns (recomputed)
23. Appendix A — findings register
24. Appendix B — debate outcomes and facts
25. Appendix C — next steps

1. Summary — what changes and why

The Phase 1 plan is sound in its shape: a .NET service on Azure App Service accepts documents from legacy MVC 4.8 applications, queues them in Azure SQL, has Azure AI Document Intelligence extract them with custom neural models, and returns canonical JSON v1 through an HMAC-signed webhook. The review kept that shape. What it found are defects in the details that would lose results, bill wrongly, or fail at deployment.

The eight changes that matter most

Change	Problem in v1	Fix in v2
High Lease token	Updates check only `Status = 'Processing'`. A worker that lost its lease still writes: it can double-POST to Azure or overwrite another worker's state.	A new `LeaseToken` on every claim; every write checks it. Zero rows = lost ownership, stop.
High Run number	A late `job.failed` from an earlier attempt can mark the retried document Failed; the later success is then ignored. The result is lost.	`RunNo` on jobs, events, results and the consumer row. Outcomes apply only to the exact bound run.
High Azure operations table	One Operation-Location column for two calls (classify, then extract). A crash mixes them up; classifier pages are lost from billing.	`AzureOperations`: one row per Azure call, written before the POST.
High Ledger per operation	Billing is written only when a job completes. Azure work billed on failed jobs never reaches the ledger. A missing price silently drops the row.	Ledger row when an Azure operation succeeds, whatever the job outcome. Missing price stops and alerts.
High Pilot upload authorization	`DocsInController.UploadFile()` has no authorization. Anyone could send billable documents.	`[Authorize]` on that one action; `Submit` refuses an anonymous user.
High SQL permissions	A pipeline with only `db_ddladmin` cannot create users or grant roles; the app's `db_datawriter` can rewrite the "append-only" ledger.	Bootstrap by an Entra admin (`WITH OBJECT_ID`); custom `docai_runtime` role with INSERT-only ledger.
High Always On + .NET 10	The worker runs inside the web app; without Always On the app unloads after ~20 idle minutes. .NET 9 support ends 10 Nov 2026.	`alwaysOn: true`; liveness health check; target .NET 10 LTS.
High Consumer dedupe	The same file on two operations returns the first operation's row; the second operation loses its link.	Consumer dedupe adds the business entity and the shipping line.

What stays exactly as designed

- Asynchronous request-reply with the SQL Jobs table as the queue, claimed with `ROWLOCK, UPDLOCK, READPAST` (an extra test runs it with Azure SQL's default `READ_COMMITTED_SNAPSHOT ON`).
- Managed identity to every Azure resource, local authentication disabled and enforced by Azure Policy.
- HMAC-SHA256 over `"{timestamp}.{rawBody}"`, two secrets during rotation, registered callback URLs (no SSRF).
- One transaction for outcome + result + ledger + webhook outbox; idempotent intake by file hash; 404 for another consumer's job.
- The kit footprint: a folder of four files in the existing web project, one table, settings, one route — no new project.

2. How the review was done

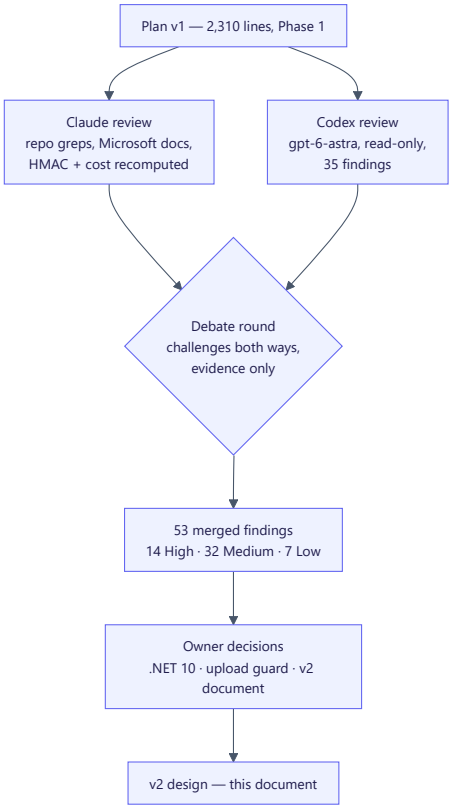


Figure 1. Two independent reviews, one debate round, one merged register

What was checked, and how

Check	Result
HMAC test vector	Secret decodes to 32 bytes; body is 326 bytes; `1793268922` = 2026-10-29 10:15:22 UTC; signature `sha256=4dad37de…3d31e14` recomputed exactly by both reviewers.
Cost examples	Example 1 ($93.43) and Example 2 ($775.31) recompute exactly from the stated inputs. Break-even of the 20,000-page tier: $540 ÷ $0.030 = 18,000 pages.
Coverage percentages	84.6 %, 88.5 %, 88.2 %, 92.9 %, 100 % — rounding correct. Several items marked "covered" were not, which is a different problem (section 22).
Repository (pilot)	Routes mapped by hand in `WebApiConfig.cs` ✓ · Hangfire 1.8.22 running through OWIN ✓ · Forms authentication, no OWIN cookie middleware ✓ · no Web API global filters or message handlers ✓ · `Defaults.UnEditableCompanyName` exists ✓ · `<httpRuntime>` has no `targetFramework` (legacy ASP.NET behaviour) ⚠ · IIS allows 4 GB request bodies ⚠ · `DocsInController.UploadFile()` is anonymous ✗
Azure facts	Delete Analyze Result exists in API `2024-11-30`; results kept 24 h from completion; S0 default 15 analyze / 50 GET requests per second; Always On available on Basic and needed for background work; App Service outbound IPs come from a shared pool and can change with the pricing tier; `CREATE USER … FROM EXTERNAL PROVIDER WITH OBJECT_ID` avoids the Directory Readers lookup; UUIDv7 does not insert in order in a SQL Server `uniqueidentifier` index.

The full register of findings is in Appendix A; disputed points and how they were settled are in Appendix B.

3. Scope (unchanged)

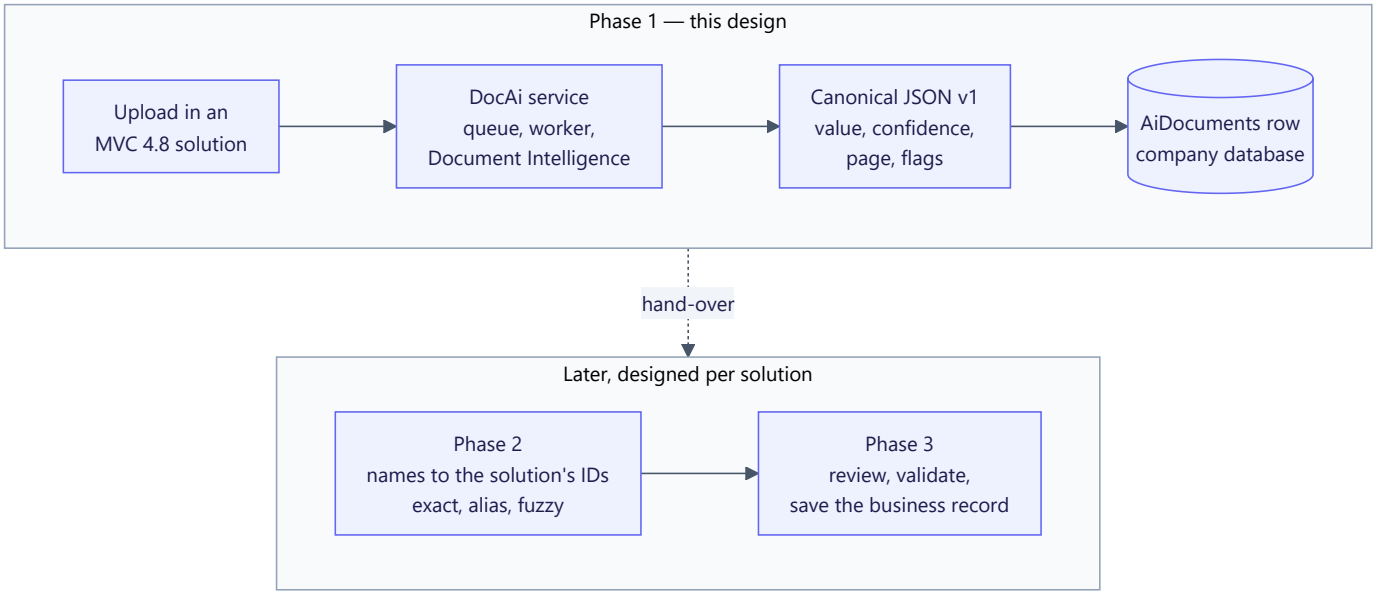


Figure 2. Phase 1 ends when the canonical JSON is stored in the company database

- **Consumer** = one deployment of one solution for one company (for example `FWD-UNF`). Cost is measured per consumer and rolled up per company and per product.
- **The five concerns** are unchanged: cost per application on a shared resource, performance, security, exception handling and logging, and no core authorization change or new project layer in the MVC 4.8 solutions.
- **Environments:** `rg-docai-dev` and `rg-docai-prod`, France Central.

4. Solution structure

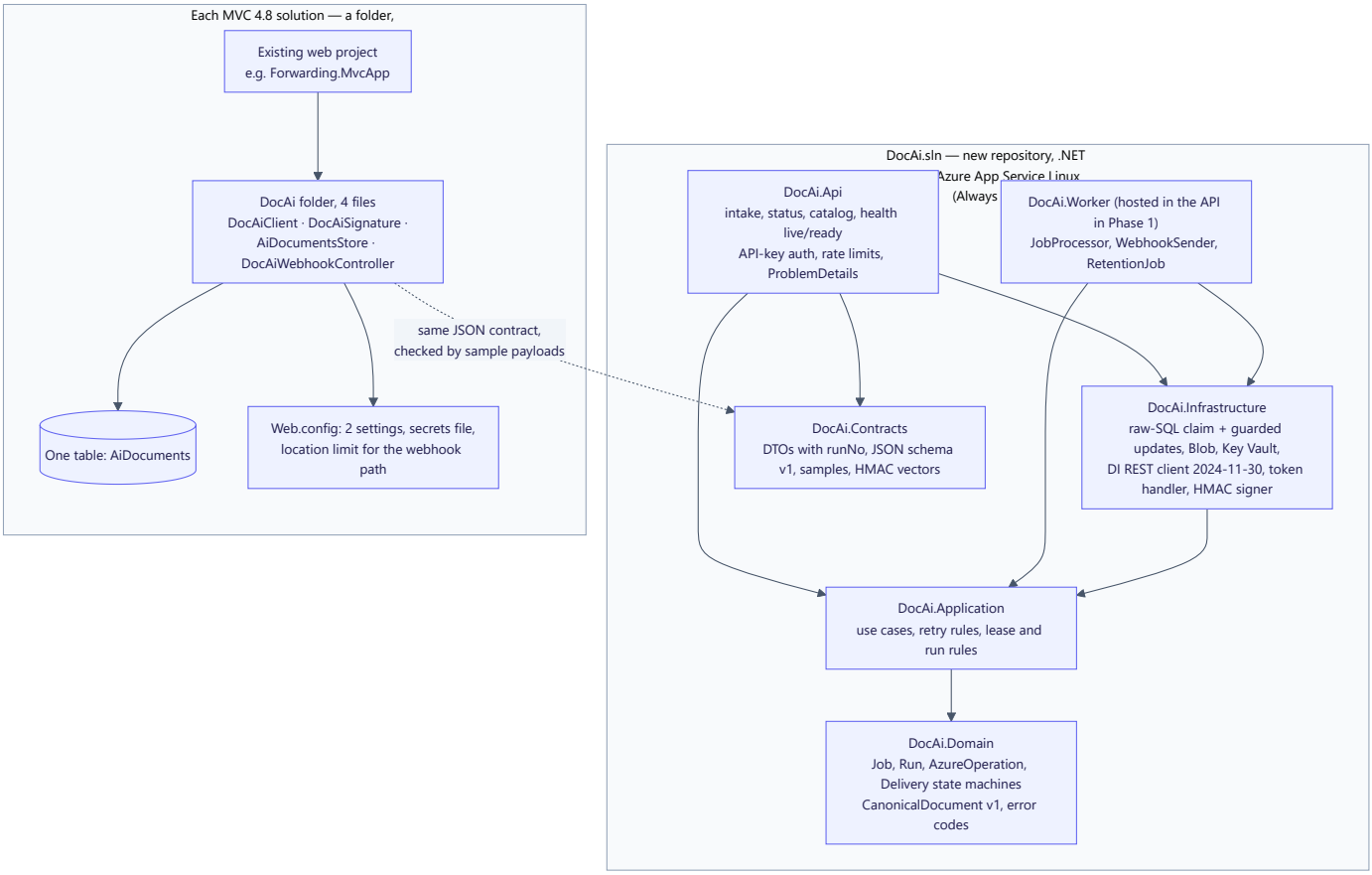


Figure 3. The service solution and the kit inside each MVC 4.8 project

Repository layout of the service

```
DocAi.sln
  src/
    DocAi.Api/              POST /api/v1/jobs, GET /api/v1/jobs/{id}, GET /api/v1/catalog,
                            /health/live (platform probe), /health/ready (alerts only)
    DocAi.Worker/           JobProcessor (claim, classify, extract, poll, complete), WebhookSender,
                            RetentionJob (a PeriodicTimer BackgroundService; Quartz not needed)
    DocAi.Application/      SubmitJob, GetJob, ProcessJob, CompleteRun, FailRun, DeliverWebhook
                            ports: IJobQueue, IJobRepository, IOperationStore, IDocumentStore,
                                   IDocumentAnalyzer, IWebhookSigner, ISecretStore, IUnitOfWork
    DocAi.Domain/           state machines (Job, Run, AzureOperation, WebhookDelivery), CanonicalDocument v1
    DocAi.Infrastructure/   SQL statements [S1]–[S19], Blob, Key Vault (1 h secret cache),
                            DocumentIntelligenceRestClient + BearerTokenHandler, HMAC signer
    DocAi.Contracts/        DTOs, JSON Schema, sample payloads, both HMAC test vectors
  db/bootstrap/             00_bootstrap_identities.sql   — Entra admin, once per environment
  db/migrations/            01_docai_schema.sql, 02_docai_grants.sql — DbUp, pipeline identity
  db/reference/             03_docai_statements.sql, 04_docai_billing.sql — not migrations
  infra/                    main.bicep (alwaysOn, healthCheckPath, policies, alerts), dev/prod params
  tests/                    Domain, Api (SQL Server container with READ_COMMITTED_SNAPSHOT ON),
                            Contracts (sample payloads), load/ (k6 or Azure Load Testing)
```

The kit inside each MVC 4.8 solution

```
<existing web project>/                     e.g. Forwarding.MvcApp — nothing else in the solution changes
  DocAi/DocAiClient.cs                      Submit (needs an authenticated user), SendPending, Refresh, GetCatalog
  DocAi/DocAiSignature.cs                   header pre-check, HMAC with two secrets, fixed-time compare, IsFresh
  DocAi/AiDocumentsStore.cs                 parameterized ADO.NET, statements [C1]–[C8]
  Controllers/DocAi/DocAiWebhookController.cs  the webhook endpoint (Web API in the pilot)
  App_Data/docai.secrets.config             API key and webhook secrets — NOT in source control
db/docai/10_company_db_schema.sql           the AiDocuments table, run on every company database
```

5. Where each part lives

Concern	Lives in	Why there	v2
Consumer authentication, rate limit, file validation, correlation-id check	`DocAi.Api`	The edge: bad input is rejected before it costs storage or Azure pages.	changed
Idempotency key and run creation (new / requeue / reprocess)	`DocAi.Application` + [S2]	Business rule decided once, under a key-range lock, in one transaction.	new
Retry policy, back-off, lease length, takeover budget	`DocAi.Application`	Plain rules, unit-tested without SQL.	changed
Lease fencing (`LeaseToken` on every write)	`DocAi.Infrastructure` statements	Every guarded statement carries the token; zero rows means ownership lost.	new
Queue claim with `READPAST`	`DocAi.Infrastructure`	SQL-specific; nothing else needs to know how a row is locked.	same
Azure call records (Sending → Accepted → Succeeded / Unknown)	`AzureOperations` table, `IOperationStore`	Survives a crash between any two steps; feeds billing and reconciliation.	new
Classifier class → shipping line	`ClassifierClasses` table	The model team maps classes when it activates a classifier; the worker never guesses.	new
Azure calls (DI REST `2024-11-30`, Blob, Key Vault)	`DocAi.Infrastructure`	One place holds the `TokenCredential`.	same
Canonical JSON v1 (+ `runNo` in the envelope)	`DocAi.Contracts`	The only thing the service and the solutions share.	changed
Billing ledger and month close	DocAi database + `04_docai_billing.sql`	The ledger row commits with the Azure operation that caused the cost.	changed
Who may write what in SQL	`02_docai_grants.sql` roles	The runtime can only append to the ledger; operators and the model team have their own roles.	new
Submitting, receiving and storing the result	The DocAi folder in each solution	Four files, one table: no project, no package, no core auth change.	changed
Sending and refreshing in the background	Hangfire, only where it already runs (pilot)	The upload request returns at once; a recurring job catches anything left Pending.	new
Webhook body limit (1 MB)	Web.config `<location path="api/DocAi/Webhook">`	IIS rejects large bodies before any code runs; the rest of the site keeps its limits.	new
Upload authorization (pilot)	`[Authorize]` on `DocsInController.UploadFile`	One attribute on one action; login, membership and global filters untouched.	new
Matching names, review, saving records	Not in Phase 1	Phases 2 and 3 start from `AiDocuments`.	same

6. Runtime view and trust boundaries

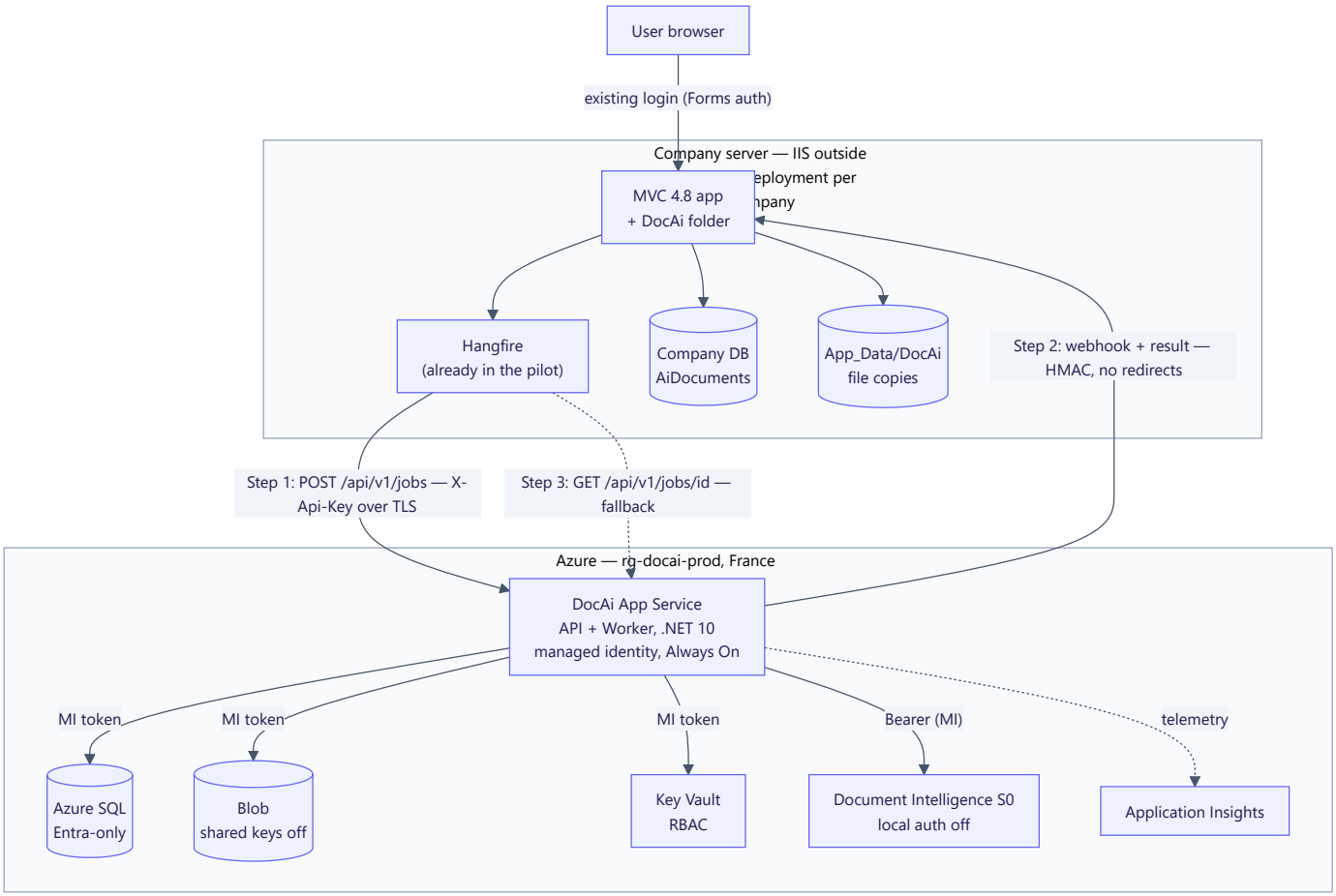


Figure 4. Who talks to whom, and how each hop authenticates

Hop	Mechanism	Secret stored where	v2 change
Browser → MVC	the solution's existing login	HttpOnly cookie	`[Authorize]` on the pilot upload action
MVC → DocAi API	per-consumer API key, `X-Api-Key`; consumer derived from the key only	Consumer: `App_Data\docai.secrets.config`. Service: SHA-256 hash only	401/403 pauses sending; never marks documents Failed
DocAi → Azure	managed identity → Entra token	nothing — there is no secret	SQL users created by an Entra admin with `WITH OBJECT_ID`
DocAi → MVC webhook	HMAC-SHA256 over `timestamp.body`, one secret per consumer, fresh timestamp on every attempt	Service: Key Vault (1 h cache). Consumer: secrets file	redirects never followed; trusted TLS chain + host name; 1 MB body limit on the consumer
Azure → DocAi	none (Azure sends nothing; the worker polls)	—	—

Outbound IP addresses. App Service outbound IPs come from a pool shared with other customers on the same scale unit, and they can change when the pricing tier changes (B1 → P0v3 is on the growth path). An IP allow-list is therefore defence in depth, not authentication — the HMAC is the authentication. Before any tier change, capture `possibleOutboundIpAddresses` and update each consumer firewall. If a fixed address is required, add a NAT Gateway with a static public IP (cost to be confirmed in the Azure pricing calculator).

7. Request cycle — part 1: upload and submit

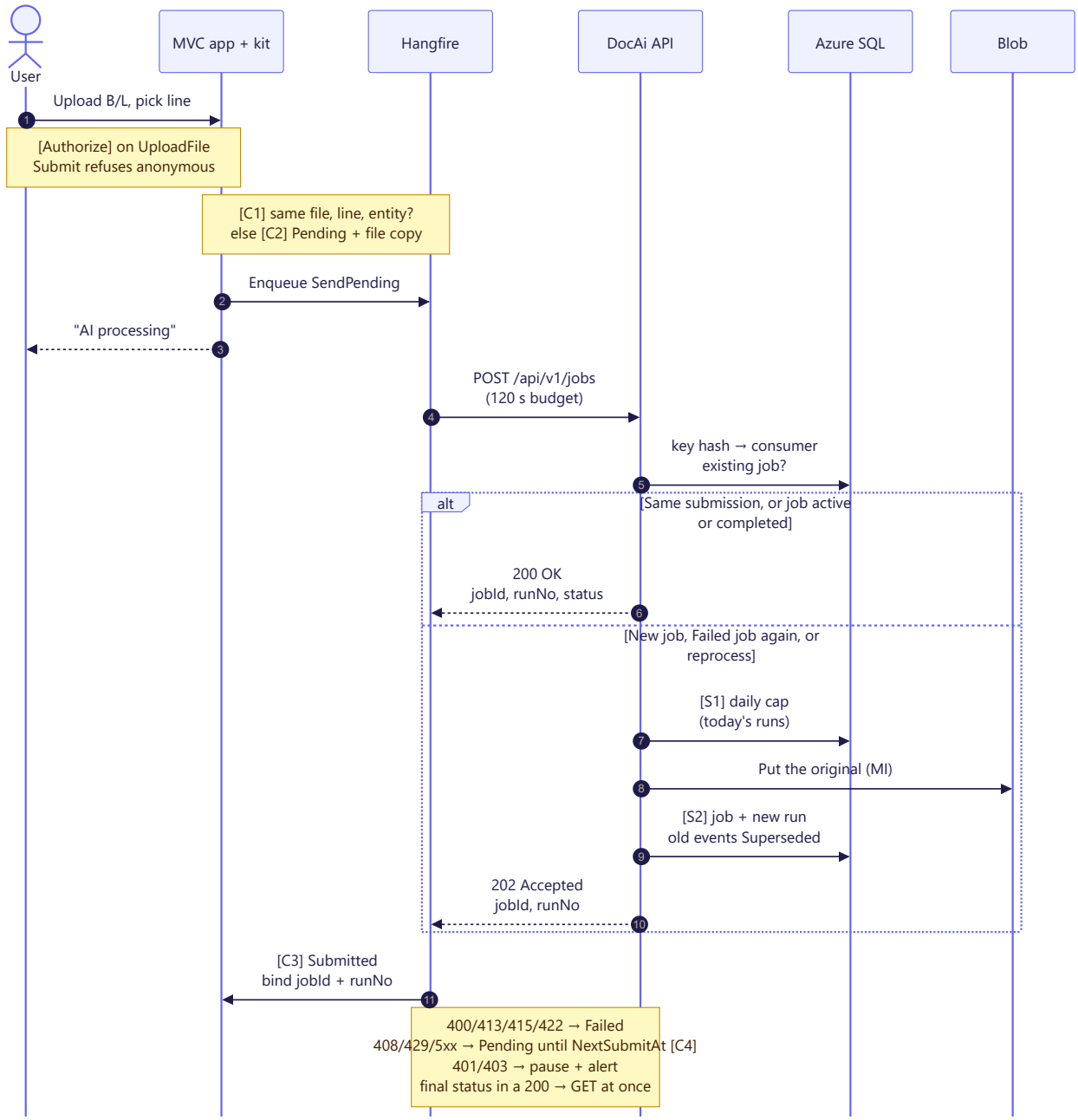


Figure 5. Steps 1–11 — the upload request returns at once; Hangfire sends the file

Steps	What	Runs in	Tables	If it fails
1	Upload, authorization, dedupe, Pending row and file copy	MVC request (sync)	AiDocuments [C1] [C2]	The user sees the error; nothing was sent.
2–3	Enqueue and return	MVC request	Hangfire	A crash before the enqueue: the recurring job finds the row still Pending after a minute.
4–10	`POST /api/v1/jobs`	Hangfire job → DocAi.Api	Jobs, JobRuns [S1] [S2], Blob	Network or 5xx: stays Pending, retried. Deadlock 1205: the whole [S2] retried with jitter (3 tries).
11	Bind the job and run	Hangfire job	AiDocuments [C3] [C4]	A webhook that arrived first already completed the row; [C3] then changes nothing.

Solutions without Hangfire call `SendPending` inline from the same request; the behaviour is identical, only slower for the user.

8. Request cycle — part 2: the worker analyzes the document

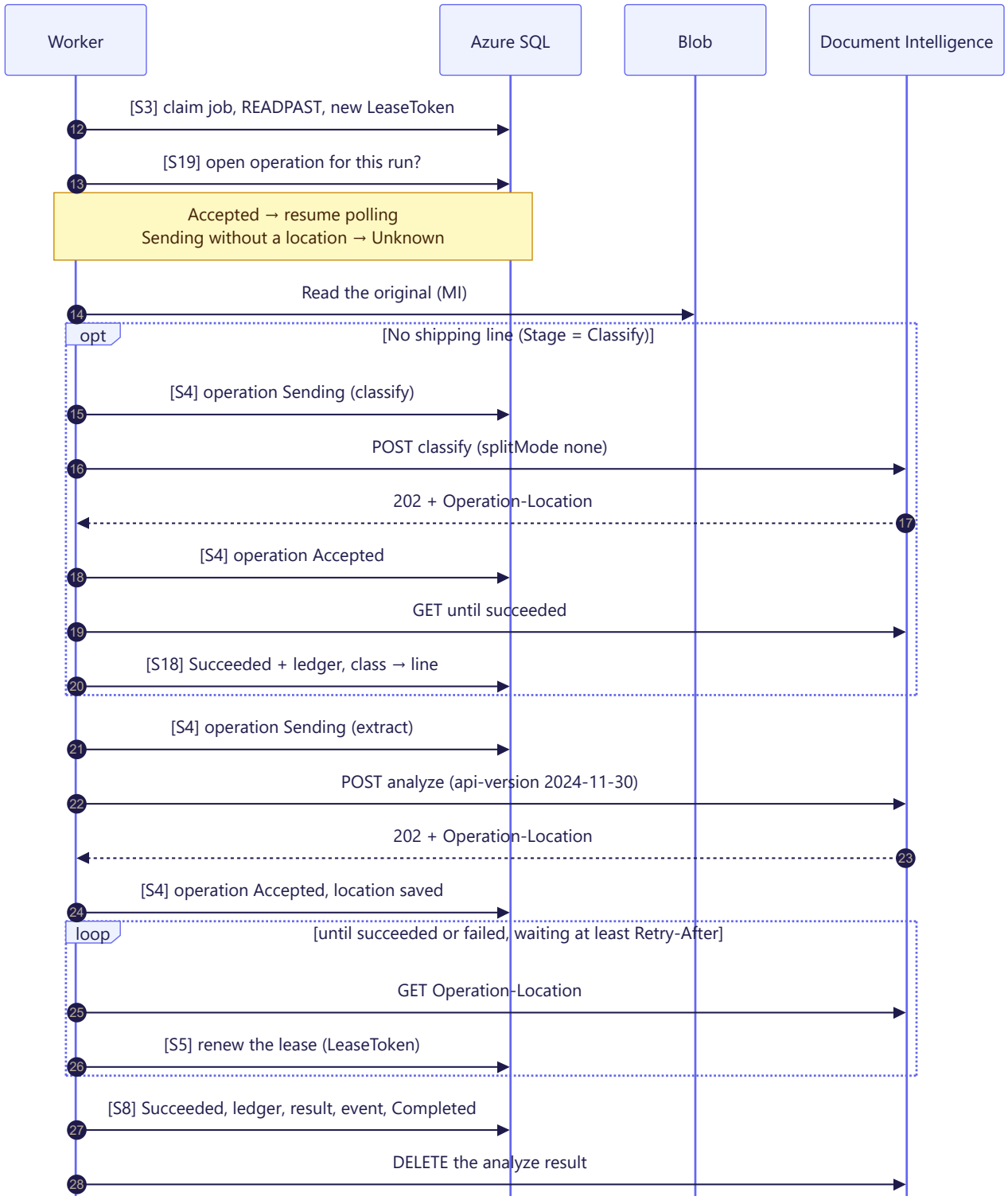


Figure 6. Steps 12–28 — every Azure call is recorded before it is made

Steps	What	Tables	If it fails
12	Claim with a new lease token	Jobs [S3]	Nothing due: sleep 2 s, backing off to 30 s. A job taken over for the 3rd time fails with `WORKER_CRASH_LOOP`.
13	Resume an open operation	AzureOperations [S19]	Accepted → resume polling. Sending without a location → marked `Unknown` (Azure may have billed it); one resubmission is allowed per run.
15–20	Classifier (only without a shipping line)	AzureOperations, UsageLedger, Jobs [S18]	Unmapped class or confidence under the threshold → Failed `UNCLASSIFIED`. The classifier pages are billed in the ledger either way.
21–24	Extraction POST	AzureOperations [S4]	See the retry rules in section 10.
25–26	Poll	Jobs [S5]	Worker crash: the lease expires after 10 min; the next worker resumes from the saved location. Lease lost: stop, write nothing.
27	Completion	5 tables [S8]	Rolled back as a whole; the job resumes from polling. A missing price stops the transaction and opens the breaker.
28	Delete the result in Azure	—	Logged only; Azure deletes it 24 h after completion anyway.

9. Request cycle — part 3: deliver and store the result

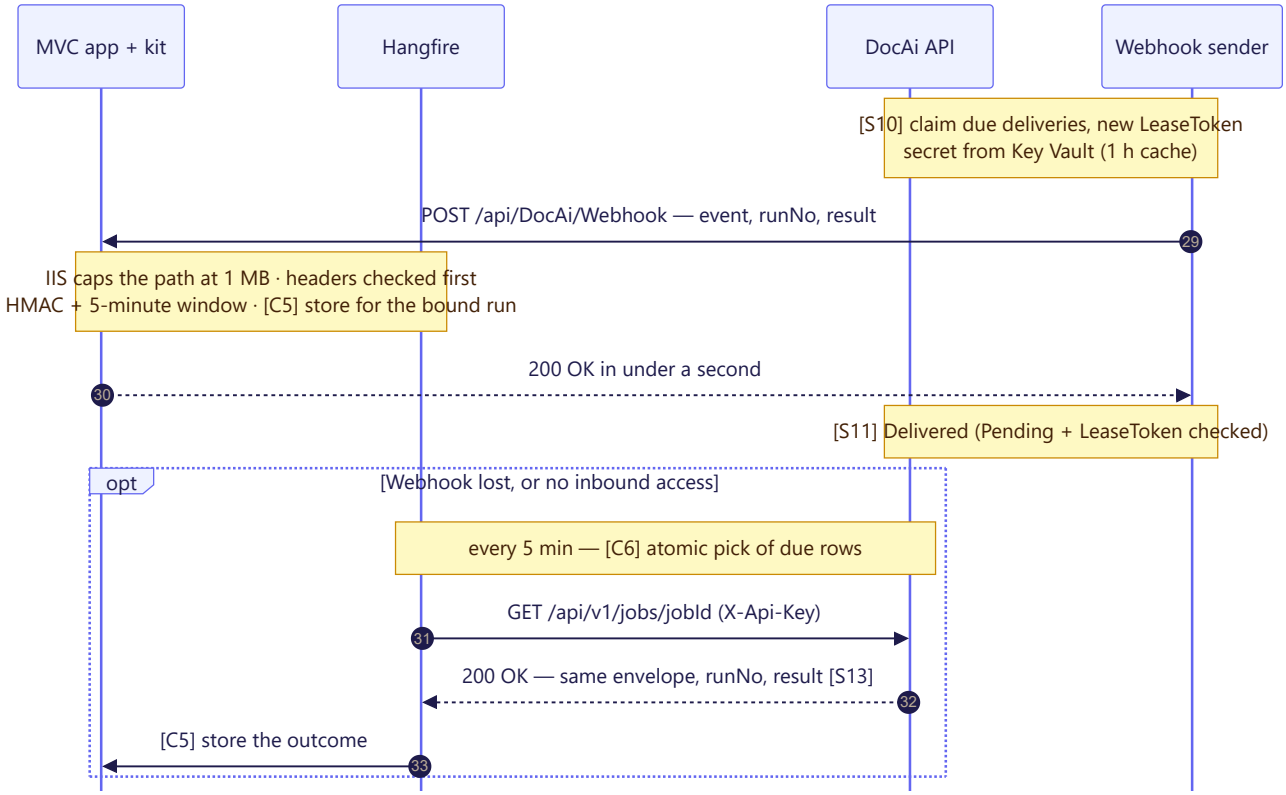


Figure 7. Steps 29–33 — the result travels in the webhook; GET is the fallback

Case	Behaviour
Consumer answers non-2xx or times out	Retries after 1, 5, 15, 60 min, then every 6 h; DeadLettered on the 8th failure; the GET fallback still gets the result. Each attempt is re-signed with a fresh timestamp.
Consumer answers 302 (login page)	Not followed. Counted as a failure — never as delivered.
Consumer answers 401	One rate-limited refresh of the secret from Key Vault, then the normal retry schedule.
Webhook arrives before the POST reply	[C5] matches by `clientRequestId` and completes the row; the late [C3] then changes nothing.
A newer run was created	Older Pending events were marked Superseded; an event already in flight is rejected by the consumer because its run does not match.

10. The worker in detail and the retry rules

The worker is one loop in three parts. Every write after the claim carries the lease token; every red box ends in one transaction, [S9], that sets the job Failed and writes a `job.failed` event for the run.

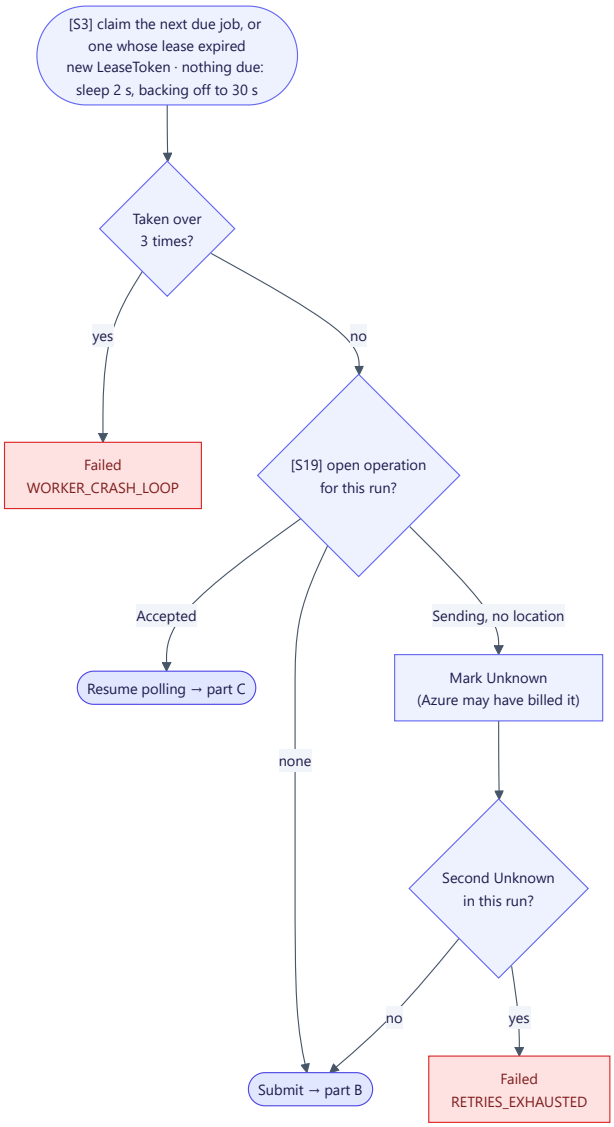


Figure 8A. Claim and resume

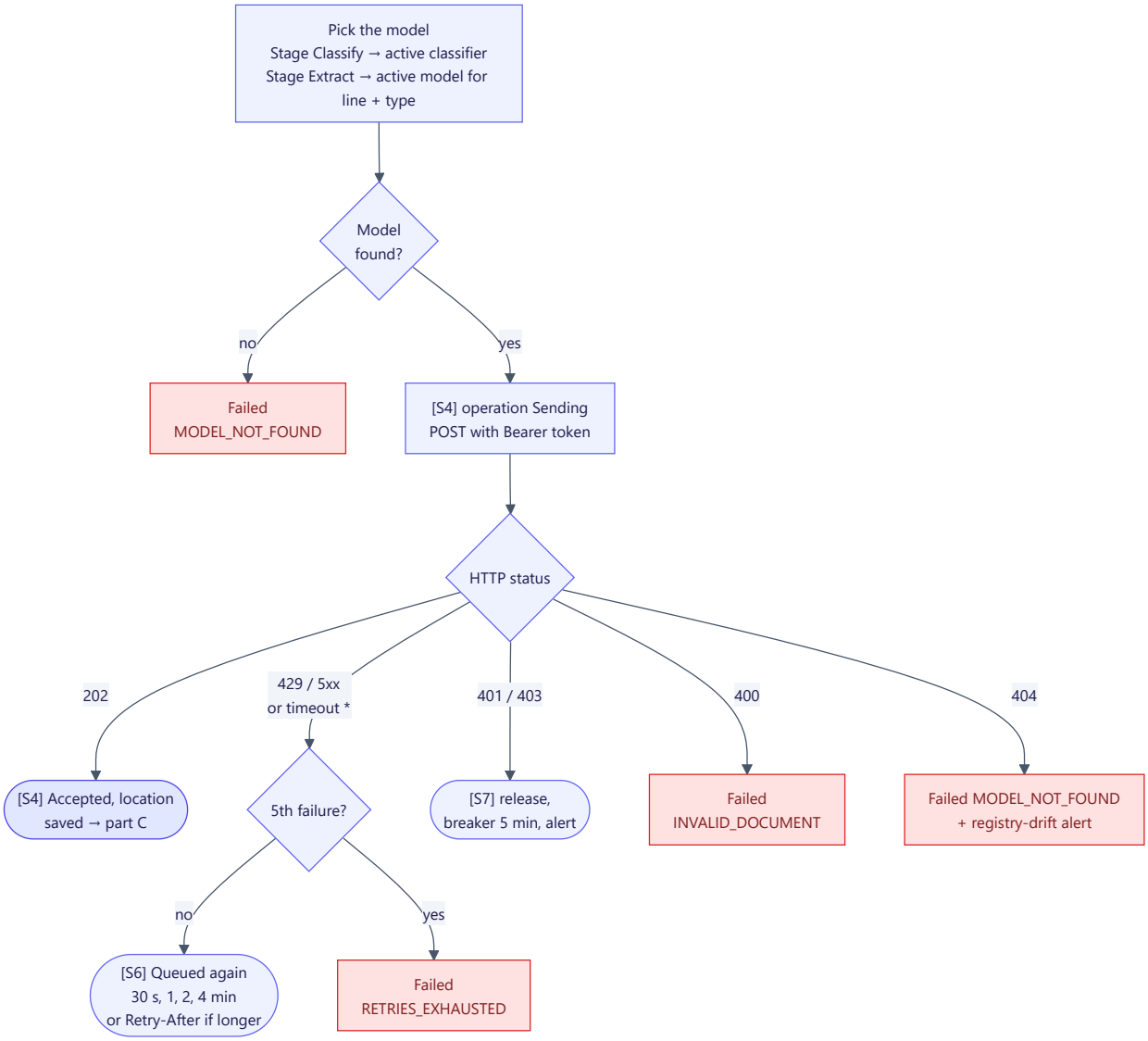


*Figure 8B. Submit to Azure — * a timeout first marks the operation Unknown*

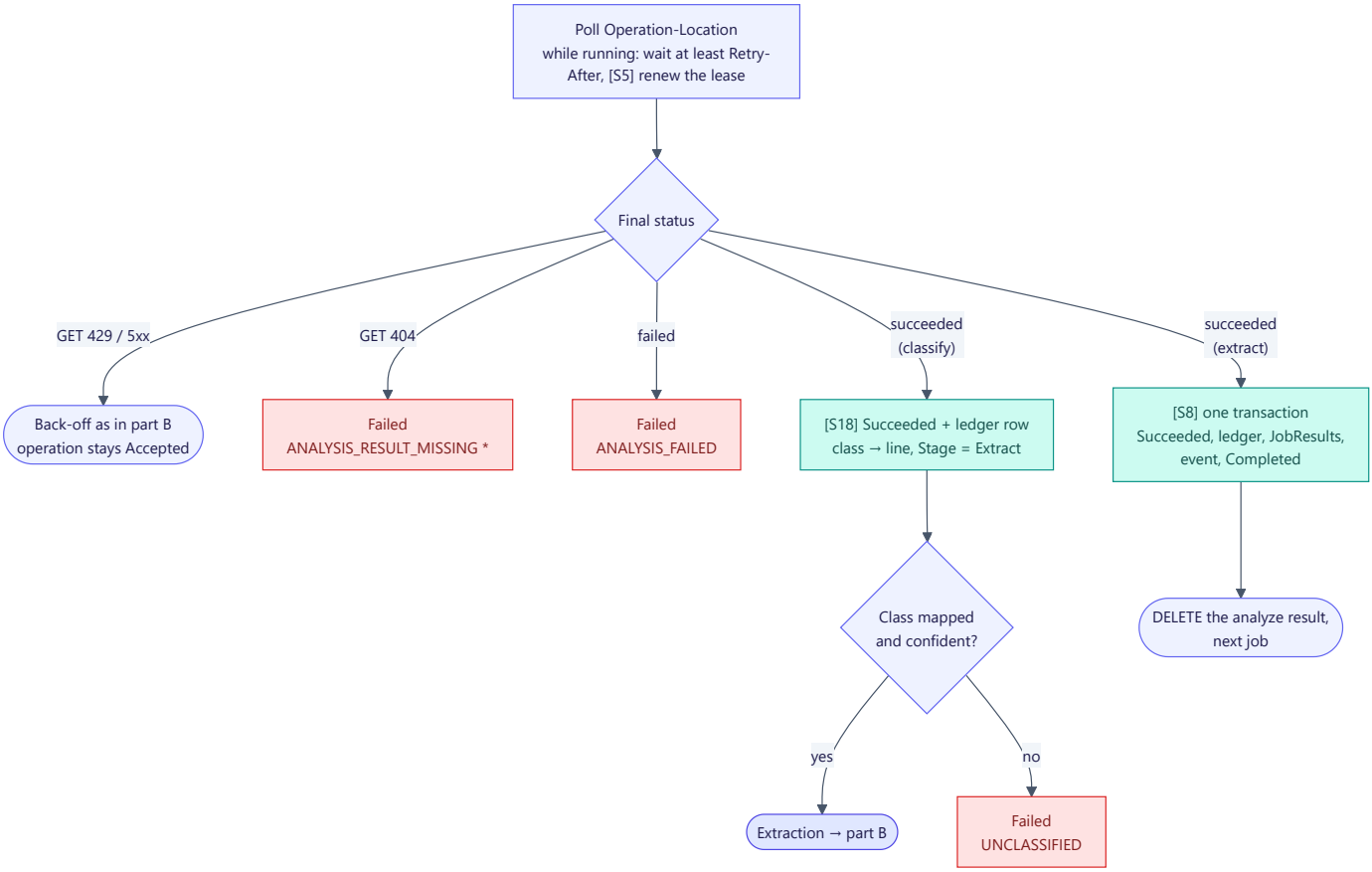


*Figure 8C. Poll and finish — * re-analyze instead only when the operation was accepted more than 24 h ago*

Retry rules in one place

Error	Where	What happens	Uses an attempt
429, 5xx on `POST analyze`	Worker	Back to Queued after 30 s, 1, 2, 4 min — or the `Retry-After` value if longer; Failed `RETRIES_EXHAUSTED` on the 5th.	yes
Timeout on `POST analyze`	Worker	Operation marked `Unknown` (Azure may have accepted and billed it), then the same back-off. A second Unknown in one run fails it.	yes
429, 5xx on the `GET` poll	Worker	Same back-off; the operation stays Accepted, so the retry resumes polling — no second POST.	yes
401 or 403 from Azure	Worker	Job released, worker paused 5 min (breaker), Critical alert.	no
No price in force for a meter	Worker	The completing transaction throws; treated like 401/403 — breaker and alert — so no completion is ever recorded without its cost.	no
400 from Azure, or the analysis ends `failed`	Worker	Failed with `INVALID_DOCUMENT` / `ANALYSIS_FAILED`, `job.failed` event.	—
404 on `POST analyze`	Worker	`MODEL_NOT_FOUND` + "registry drift" alert: the registry names a model that Azure does not have.	—
404 on the `GET` poll	Worker	If the operation was accepted more than 24 h ago the result has expired: start again (billed twice; reconciliation shows it). Otherwise `ANALYSIS_RESULT_MISSING` + alert, no paid retry.	yes / —
Lease expired, job taken over	Worker	`TakeoverCount + 1`; the 3rd takeover fails `WORKER_CRASH_LOOP` (a document that crashes the process cannot loop forever).	no
Lease lost (another worker holds the token)	Worker	The guarded update changes 0 rows: stop at once, write nothing, take the next job.	no
Webhook non-2xx, timeout or redirect	Sender	1, 5, 15, 60 min, then every 6 h; DeadLettered on the 8th.	—
DocAi API 408, 429, 5xx or network	Kit	Row stays Pending; sent again at `NextSubmitAt` (Retry-After) by the recurring job; after 10 tries Failed `SUBMIT_EXHAUSTED`.	—
DocAi API 401 or 403	Kit	Sending paused 5 min, critical trace; documents are not failed.	—

> **Throttling.** Separate token buckets: analyze POSTs at most 10 per second (Azure S0 default: 15) and polls at most 40 per second (default: 50). `Retry-After` is always a minimum, never capped below the server's value.

11. State machines

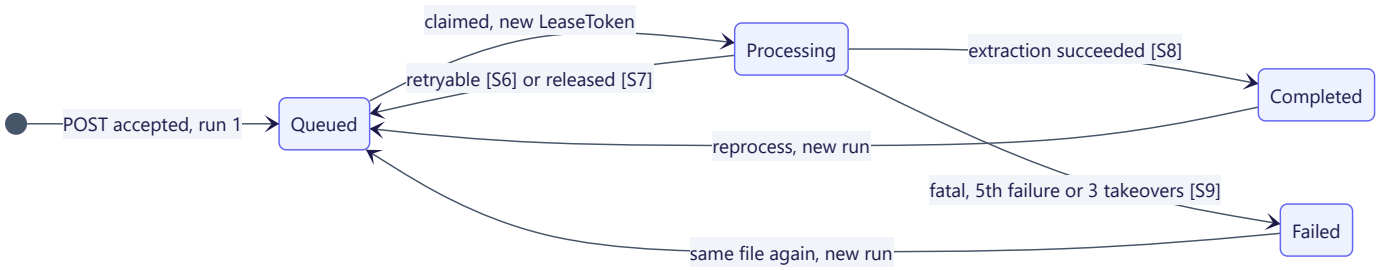


Figure 9. Job (DocAi database)

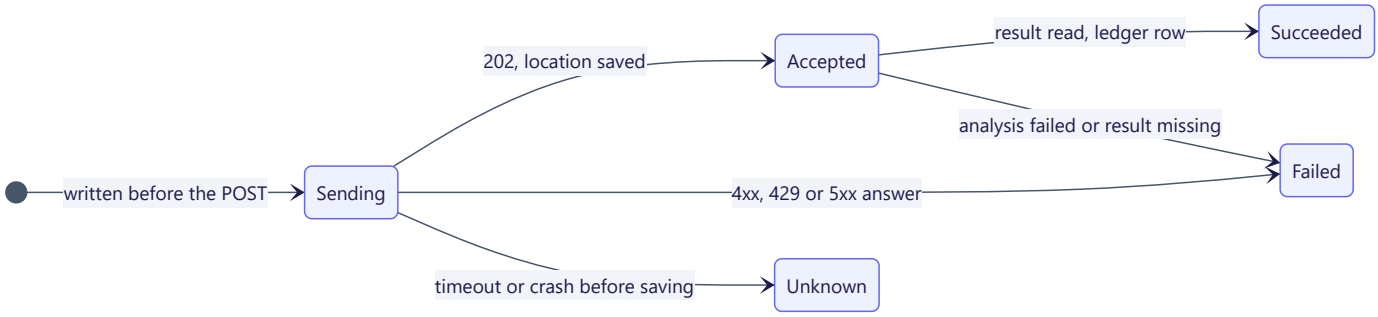


Figure 10. Azure operation (new)

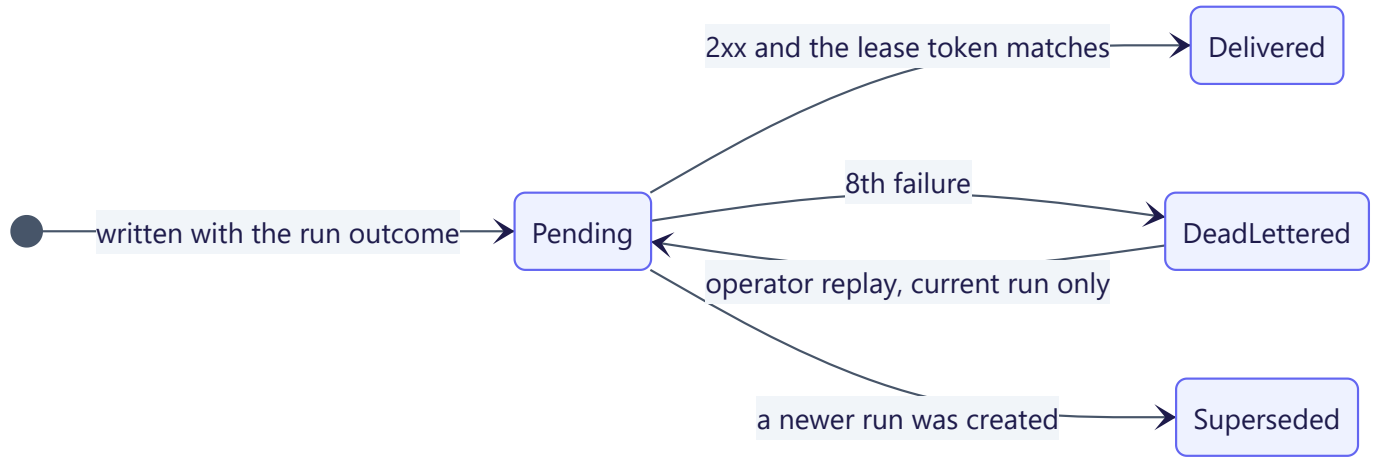


Figure 11. Webhook delivery (DocAi database)

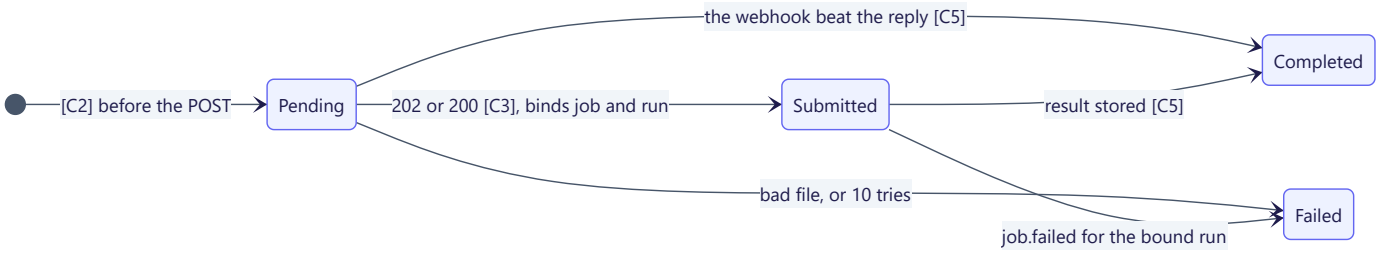


Figure 12. AiDocuments (company database)

- **A run** is one authorized analysis of a job: the first submission (run 1), the same file sent again after a failure, or an explicit `reprocess=true`. Transport retries, back-offs and lease takeovers never create a run.
- **A retry from the user is a new AiDocuments row.** [C1] skips Failed rows, so submitting again creates a new row and — on the service — a new run of the existing job. Terminal rows are never reopened.
- **Correcting a wrong shipping line**: the requested line is part of the idempotency key, so a different line is a different job and is analyzed again (billed once more, by design).

12. The two concurrency fixes: lease token and run number

12.1 Lease token — a worker that lost its lease changes nothing

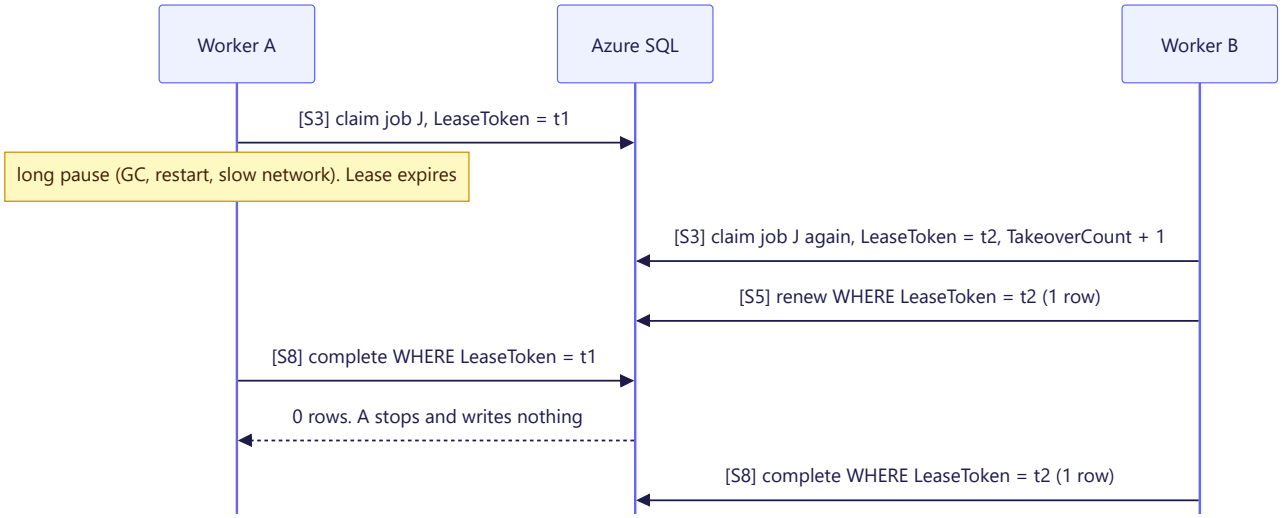


Figure 13. In v1 both workers matched Status = 'Processing'; in v2 only the current token matches

```
-- The guard every write after the claim carries ([S4]–[S9], [S18])
WHERE JobId = @jobId
  AND Status = 'Processing'
  AND LeaseToken = @leaseToken;          -- 0 rows: ownership lost — stop, never run [S9]
```

Decision from the debate: the token alone fences. Adding `LeaseExpiresAtUtc > now` would make a slow but sole owner abandon work that nobody else has taken.

12.2 Run number — a stale event never overwrites a newer run

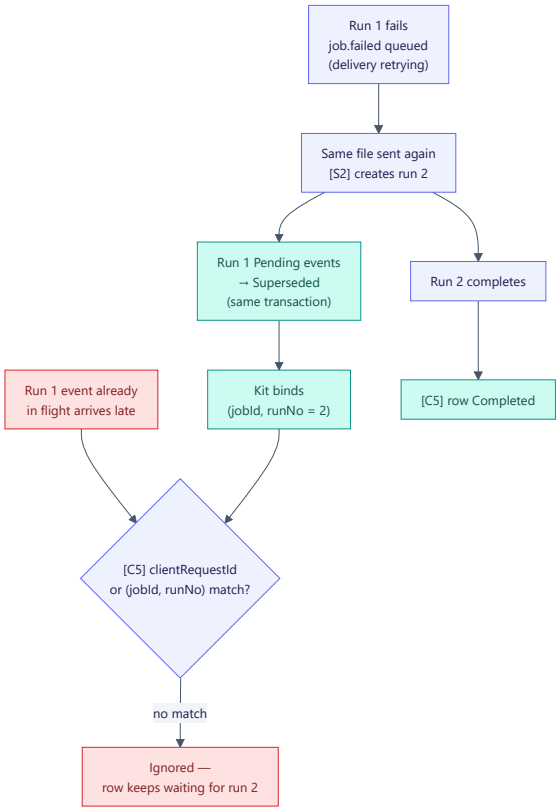


Figure 14. In v1, [C5] matched on RemoteJobId alone, so the late run 1 failure marked the new row Failed and the result was lost

```
-- [C5] v2: store an outcome only for the submission it belongs to
UPDATE dbo.AiDocuments
SET Status = CASE WHEN @status = 'Completed' THEN 'Completed' ELSE 'Failed' END,
    RemoteJobId = COALESCE(RemoteJobId, @remoteJobId),
    RemoteRunNo = COALESCE(RemoteRunNo, @runNo),
    ResultJson = @body, ErrorCode = @errorCode, ErrorMessage = @errorMessage, CompletedAt = GETDATE()
WHERE Status IN ('Pending', 'Submitted')                              -- terminal rows never change
  AND (   LocalId = @clientRequestId                                  -- our own submission
       OR (RemoteJobId = @remoteJobId AND RemoteRunNo = @runNo));     -- or a row bound to this exact run
```

13. Database — DocAi (Azure SQL, Entra-only)

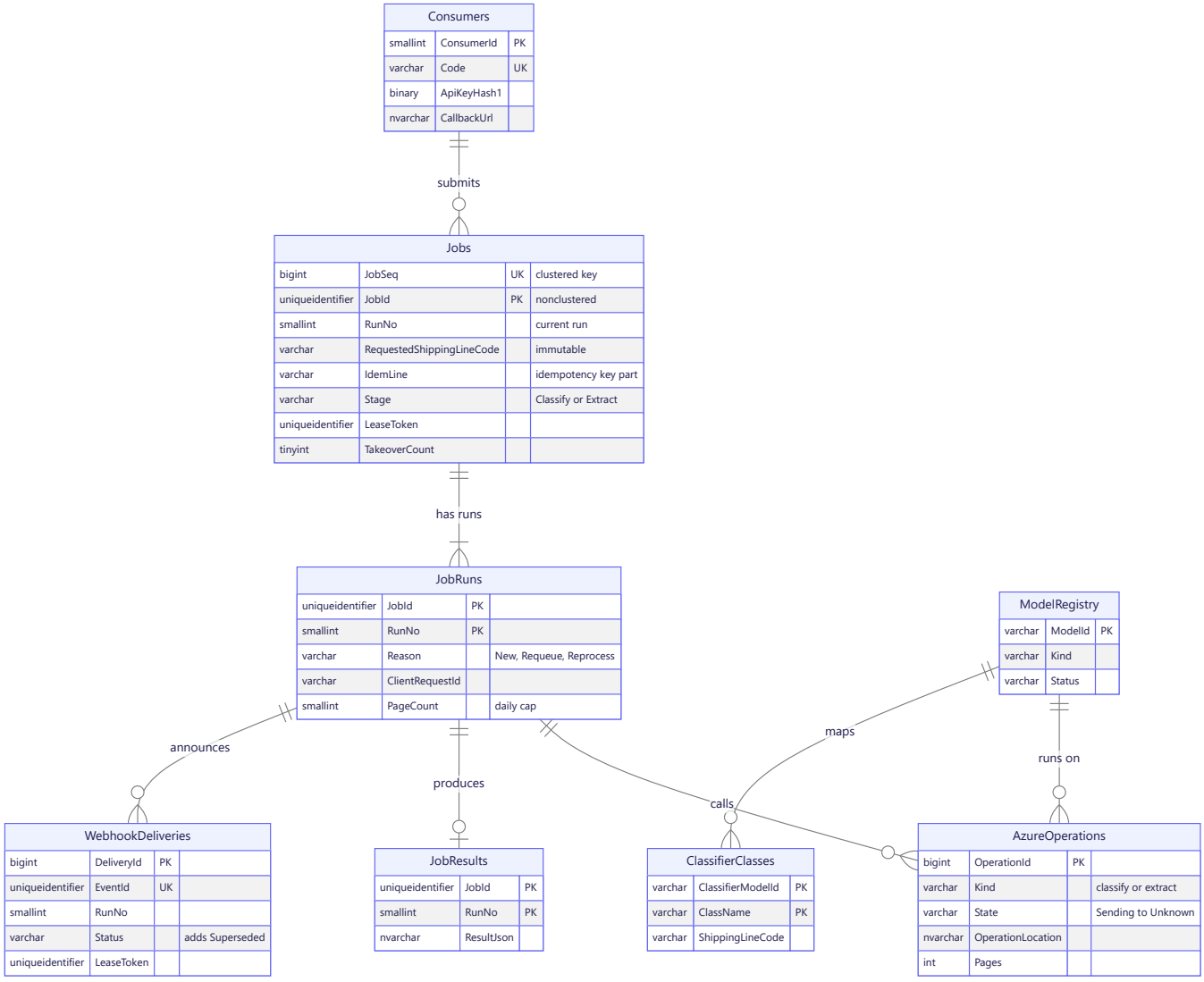


Figure 15A. DocAi database v2 — processing tables (keys and the columns that change)

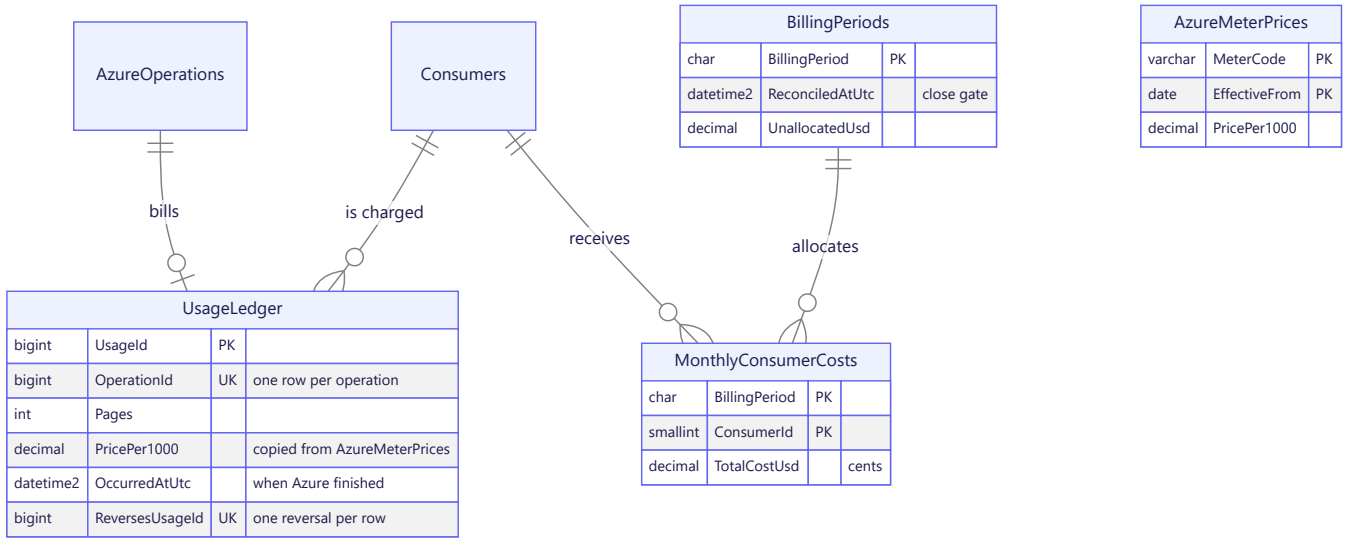


Figure 15B. DocAi database v2 — billing tables (prices are copied into each ledger row, not joined)

Table catalogue v2

Table	Purpose	What changes	Kept
Consumers	Who may call; callback URL; limits	same optional private CA reference for the callback	Always
ModelRegistry	Line + document type → model; thresholds	changed classifier rows use `MinFieldConfidence` as the class threshold; activation derives its group from the new model	Always
ClassifierClasses	Classifier class name → shipping line	new	Always
Jobs	The request, its state, the queue	changed `JobSeq` clustered, `RunNo`, `RequestedShippingLineCode` + `IdemLine`, `Stage`, `LeaseToken`, `TakeoverCount`; Operation-Location moves out; `ErrorMessage` 400 chars; `FileName` cleared by retention	Soft-deleted after 13 months
JobRuns	One row per authorized analysis	new daily cap counts these; unique `(JobId, ClientRequestId)` makes a repeated POST return the same run	With the job
AzureOperations	One row per Azure call	new at most one open operation per run and kind (filtered unique index)	Always (billing audit)
JobResults	Canonical JSON v1	changed key `(JobId, RunNo)`	13 months
WebhookDeliveries	Outbox for callbacks	changed `RunNo`, immutable `EventId`, `LeaseToken`, status `Superseded`, `PayloadJson` `nvarchar(max)`, unique `(JobId, RunNo, EventType)`; no MERGE re-arm	90 days after delivery
AzureMeterPrices	List prices by date	same	Always
UsageLedger	Billing facts, append-only	changed one row per Azure operation; `OccurredAtUtc` = when Azure finished; one reversal per row; INSERT-only for the app	Always
BillingPeriods	The month's actual Azure numbers	changed `ReconciledAtUtc` (gate for closing), `UnallocatedUsd`	Always
MonthlyConsumerCosts	Frozen monthly allocation	changed amounts in cents (decimal(12,2)), largest-remainder rounding	Always

Key design decisions (corrected)

- **Clustered key.** `JobSeq bigint IDENTITY` is the clustered key; `JobId` (any GUID) is the nonclustered primary key. SQL Server sorts `uniqueidentifier` by its last bytes first, so a UUIDv7 key does not append — v1's claim is removed.
- **Idempotency key** = `(ConsumerId, DocumentTypeCode, FileSha256, IdemLine)`, filtered on `IsDeleted = 0`. `IdemLine` is a plain column with `CHECK (IdemLine = COALESCE(RequestedShippingLineCode, '*'))` (filtered indexes and computed columns do not mix well).
- **Lease consistency:** `CHECK ((Status = 'Processing' AND LeaseToken IS NOT NULL) OR (Status <> 'Processing' AND LeaseToken IS NULL))`.
- **Queue index** filter: `WHERE Status IN ('Queued','Processing') AND IsDeleted = 0`, so the claim never needs a lookup.
- **Blob names** use the upload id and the real extension: `documents/{consumer}/{yyyy}/{MM}/{uploadId}.{pdf|jpg|png|tif}`.

Statement map — DocAi

Label	Statement	v2 change
[S1]	Daily page cap	Sums `JobRuns` created today, so requeues and reprocesses count. Runs only after the duplicate lookup.
[S2]	Intake: create, requeue or reprocess	Same submission → same run; Failed job → new run; `reprocess=true` on a Completed job → new run; older Pending events Superseded; whole transaction retried on deadlock 1205.
[S3]	Claim the next job	New `LeaseToken`; `TakeoverCount + 1` when taking over an expired lease. Hints unchanged: `ROWLOCK, UPDLOCK, READPAST`.
[S4]	Azure operation: Sending / Accepted / Failed / Unknown	New: written before the POST, updated after the reply. Replaces v1's "save Operation-Location".
[S5]	Renew the lease	Guarded by the token.
[S6]	Retryable error	Guarded; back-off = max(15 × 2^n s, Retry-After). Exhaustion decided in code, not by "0 rows".
[S7]	Release without using an attempt	Guarded; used for 401/403, missing price and graceful shutdown.
[S8]	Completion	Operation Succeeded + ledger row (price must exist, else THROW) + JobResults + immutable event + Completed, one transaction.
[S9]	Failure	Guarded; inserts a new immutable `job.failed` event for this run (no MERGE re-arm); message cut to 400 chars.
[S10]–[S12]	Webhook claim, attempt, replay	Claim sets a token (lease 5 min); the attempt checks `Status = 'Pending'` + token; replay clears the lease and only targets the current run.
[S13]	GET a job	Returns `runNo` and the result of the current run.
[S14]	Model activation	Locks the new model, derives its group from it, then retires and activates; a classifier needs its class map first.
[S15]	Retention	Results by their own age; `FileName` cleared on soft delete; Superseded deliveries purged; deletes in batches.
[S16]–[S17]	Catalog; result for a webhook	Catalog says whether auto-detect (classifier) exists per document type; result read for the event's run.
[S18]	Classifier succeeded	New: operation Succeeded + ledger row + mapped line + `Stage = 'Extract'`, one transaction.
[S19]	Open operations of the current run	New: the worker's resume point after a crash.

```
-- [S2] core of the v2 intake decision (inside SET XACT_ABORT ON / BEGIN TRANSACTION)
SELECT @existingJobId = JobId, @existingStatus = Status, @existingRun = RunNo
FROM dbo.Jobs WITH (UPDLOCK, HOLDLOCK)                -- key-range lock serializes the same file
WHERE ConsumerId = @consumerId AND DocumentTypeCode = @documentTypeCode
  AND FileSha256 = @fileSha256 AND IdemLine = @idemLine AND IsDeleted = 0;

IF @existingJobId IS NULL                                    -> insert Jobs + JobRuns (run 1)          'Created'   202
ELSE IF the run with this ClientRequestId exists             -> return that run                       'Existing'  200
ELSE IF @existingStatus = 'Failed'
     OR (@reprocess = 1 AND @existingStatus = 'Completed')   -> insert JobRuns (run n+1), reset Jobs,
                                                                UPDATE WebhookDeliveries SET Status = 'Superseded'
                                                                WHERE JobId = @existingJobId AND RunNo < @newRun
                                                                  AND Status = 'Pending'                 'Requeued'  202
ELSE                                                         -> return the current run                'Existing'  200
```

14. Database — company side (one table per company database)

The AI service never connects to a company database; the consumer never connects to DocAi's. Each company database gets one table, `AiDocuments`, with no foreign keys to legacy tables.

Column	Meaning	v2
LocalId (PK)	Sent as `clientRequestId`	same
RemoteJobId, **RemoteRunNo**	The exact job and run this row is bound to	run new
Status	Pending · Submitted · Completed · Failed	same
DocumentTypeCode, ShippingLineCode	From the catalog; the line is part of the dedupe	changed
EntityName, EntityID	What the document belongs to (`'Operation'` , `Operations.ID`); part of the dedupe	changed
Reprocess	The user asked for a new billed analysis	new
StoredFileName, OriginalFileName, FileSha256, PageCount	The kit's copy and file facts	same
ResultJson, ErrorCode, ErrorMessage (500)	The body as received; message at least as long as the service's (400)	same
SubmitAttempts, **NextSubmitAt**	Retry budget (10) and the Retry-After time	new
LastCheckedAt	Throttles the GET fallback	same
StoredFileDeletedAt, ResultPurgedAt	Housekeeping and retention done	new

Statement map — company database

Label	Statement	v2 change
[C1]	Same document already sent?	Matches file + type + **line + entity**; skips Failed rows; skipped entirely for `reprocess` .
[C2]	New Pending row before the POST	Adds `Reprocess` .
[C3]	Service answered 202/200	Only from `Pending` ; binds `RemoteJobId` + `RemoteRunNo` ; never downgrades a terminal row.
[C4]	Send failed	Classified by status (permanent / retry later / pause); `NextSubmitAt` from Retry-After; 10 tries → Failed `SUBMIT_EXHAUSTED` ; 401/403 do not use a try.
[C5]	Store an outcome	Matches `LocalId` or the exact `(jobId, runNo)` ; terminal rows never change (figure 14).
[C6]	Pick due rows for resend / refresh	One atomic `UPDATE … OUTPUT` (by row, or a batch for the recurring job) replaces the select + unconditional update.
[C7]	Delete the kit's file copy	Marks `StoredFileDeletedAt` ; 7 days after a result, 30 days after a failure.
[C8]	Result retention	New: clears `ResultJson` after the company's period (default 13 months, setting `DocAi:ResultRetentionMonths`). Phase 2 may keep data longer by its own policy.

```
-- [C6] v2: claim due work atomically (two page views or two jobs can never both act on one row)
UPDATE TOP (@batchSize) dbo.AiDocuments
SET LastCheckedAt = GETDATE()
OUTPUT inserted.LocalId, inserted.Status, inserted.RemoteJobId, inserted.RemoteRunNo
WHERE (@localId IS NULL OR LocalId = @localId)
  AND (   (Status = 'Pending'   AND CreatedAt   < DATEADD(minute,  -1, GETDATE())
                                AND (NextSubmitAt IS NULL OR NextSubmitAt <= GETDATE()))
       OR (Status = 'Submitted' AND SubmittedAt < DATEADD(minute, -10, GETDATE())))
  AND (LastCheckedAt IS NULL OR LastCheckedAt < DATEADD(minute, -1, GETDATE()));
```

15. Database permissions

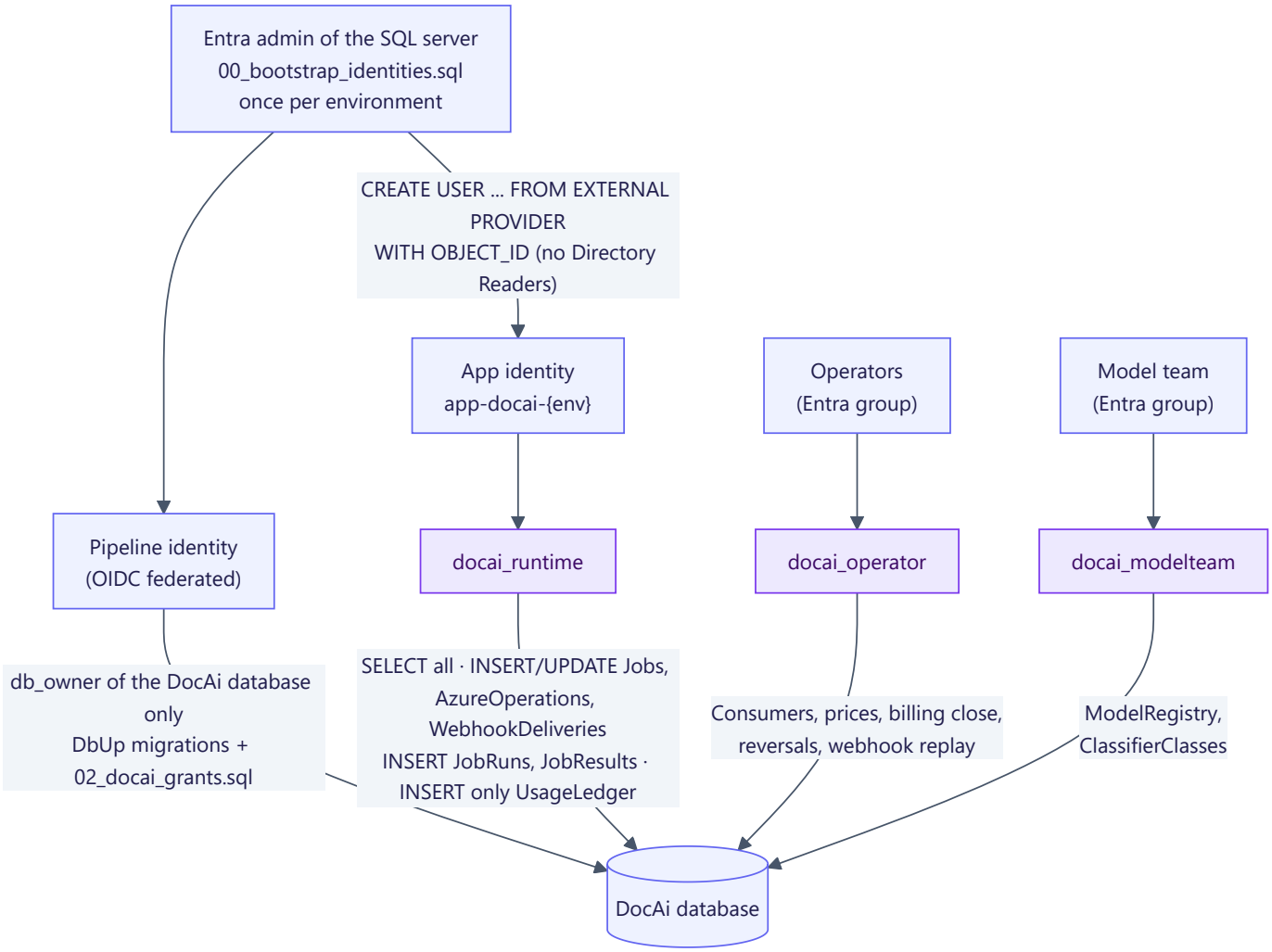


Figure 16. Who may write what — the runtime can only append to the ledger

```
-- 00_bootstrap_identities.sql — run by a Microsoft Entra admin of the server (sqlcmd variables per environment)
CREATE USER [$(AppIdentityName)]      FROM EXTERNAL PROVIDER WITH OBJECT_ID = '$(AppPrincipalId)';
CREATE USER [$(PipelineIdentityName)] FROM EXTERNAL PROVIDER WITH OBJECT_ID = '$(PipelinePrincipalId)';
ALTER ROLE db_owner ADD MEMBER [$(PipelineIdentityName)];      -- this database only; no server role

-- 02_docai_grants.sql — run by the pipeline after the schema migration
CREATE ROLE docai_runtime;
GRANT SELECT ON SCHEMA::dbo                           TO docai_runtime;
GRANT INSERT, UPDATE ON dbo.Jobs                      TO docai_runtime;
GRANT INSERT ON dbo.JobRuns                           TO docai_runtime;
GRANT INSERT, UPDATE ON dbo.AzureOperations           TO docai_runtime;
GRANT INSERT, DELETE ON dbo.JobResults                TO docai_runtime;   -- DELETE: retention [S15]
GRANT INSERT, UPDATE, DELETE ON dbo.WebhookDeliveries TO docai_runtime;  -- DELETE: retention [S15]
GRANT INSERT ON dbo.UsageLedger                       TO docai_runtime;   -- append only: no UPDATE, no DELETE
ALTER ROLE docai_runtime ADD MEMBER [$(AppIdentityName)];
-- Nothing on Consumers, ModelRegistry, ClassifierClasses, AzureMeterPrices, BillingPeriods, MonthlyConsumerCosts.
```

v1 ran `CREATE USER` and `ALTER ROLE` inside a migration executed by a `db_ddladmin` identity, which lacks the permission for both, and hard-coded `app-docai-prod`, which breaks the dev environment.

16. Cost per application — model and formula

Azure bills the Document Intelligence **resource**, not the caller. With every consumer on one shared resource, the invoice says how many custom pages were analyzed, not who sent them, and tags cannot split one resource's meter. So the service records every billable Azure operation itself — in v2, at the moment the operation succeeds.

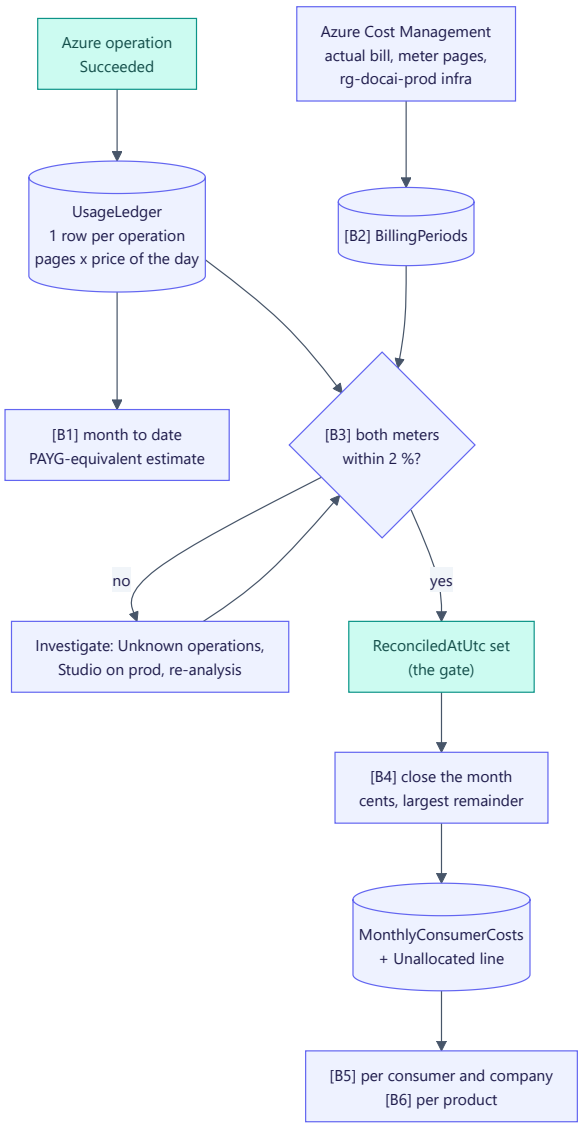


Figure 17. From an Azure operation to the monthly statement

What is billed

Item	Price	When	How it reaches a consumer
Custom extraction `DI_CUSTOM_EXTRACT`	$30 per 1,000 pages ($0.030)	every successful extraction	ledger row: pages Azure analyzed
Custom classification `DI_CUSTOM_CLASSIFY`	$3 per 1,000 pages ($0.003)	only without a shipping line	ledger row — now also when the job later fails
Commitment tier (optional)	$540/month for 20,000 pages, then $27 per 1,000	after 2–3 months above ~18,000 pages	month end, by page share
Infrastructure	Phase 1 ≈ $32.23 · growth ≈ $100.99	always	month end, by page share
Model development (dev resource)	training above 10 free hours at $3/h	while training	month end, by page share

Prices: Azure Retail Prices API, France Central, USD, read 11 October 2026 (from v1; not re-verified in this review — re-check before go-live and add a new `AzureMeterPrices` row when one changes). Phase 1 infra = B1 $13.14 + SQL S0 $18.40 + 20 GB blob $0.38 + Key Vault $0.30. Growth infra = P0v3 $63.51 + SQL S1 $36.80 + blob $0.38 + Key Vault $0.30 = **$100.99** (v1 left storage and Key Vault out and showed $100.31).

The formula (unchanged in shape)

```
E(c), C(c)   extraction and classification pages of consumer c in the ledger;   E, C = totals of all consumers
s(c)         = E(c) / E                                          the consumer's page share
BillE        actual extraction cost of the shared resource
             pay-as-you-go: pages x $0.030        20,000-page tier: $540 + pages above 20,000 x $0.027
BillC        actual classification cost;   Infra = rest of rg-docai-prod;   ModelDev = the dev DI resource

Cost(c)      = BillE x s(c)  +  BillC x C(c) / C  +  (Infra + ModelDev) x s(c)
Sum of Cost(c) = BillE + BillC + Infra + ModelDev      — now guaranteed to the cent (section 17.3)
```

17. Cost calculations — worked examples

17.1 Example 1 — pilot month, pay-as-you-go, three consumers

Consumer	Extraction pages	Classification pages	Share	Extraction	Classification	Infra share	Total	Per extraction page
FWD-UNF	1,200	400	60 %	$36.00	$1.20	$19.34	$56.54	$0.0471
FWD-CPW	500	0	25 %	$15.00	$0.00	$8.06	$23.06	$0.0461
YARD-UNF	300	0	15 %	$9.00	$0.00	$4.83	$13.83	$0.0461
Total	**2,000**	**400**	**100 %**	**$60.00**	**$1.20**	**$32.23**	**$93.43**	**$0.0467**

Extraction: 2,000 × $0.030 = $60.00. Classification: 400 × $0.003 = $1.20 (FWD-UNF uploaded 400 pages without choosing the line). Infra $32.23 = 3,223 cents: exact shares 1,933.8 / 805.75 / 483.45 cents → whole cents 1,933 / 805 / 483 = 3,221 → the 2 leftover cents go to the largest fractions (.80 and .75) → 19.34 / 8.06 / 4.83. Same as v1, now by rule.

17.2 Example 2 — growth month, 20,000-page tier, six consumers (corrected infra)

Consumer	Extraction pages	Share	Extraction (tier)	Infra share	Total	Per page
FWD-UNF	6,000	24 %	$162.00	$24.23	$186.23	$0.0310
FWD-CPW	4,000	16 %	$108.00	$16.16	$124.16	$0.0310
FWD-T3	5,000	20 %	$135.00	$20.20	$155.20	$0.0310
FWD-T4	3,000	12 %	$81.00	$12.12	$93.12	$0.0310
YARD-UNF	4,000	16 %	$108.00	$16.16	$124.16	$0.0310
YARD-CPW	3,000	12 %	$81.00	$12.12	$93.12	$0.0310
Total	**25,000**	**100 %**	**$675.00**	**$100.99**	**$775.99**	**$0.0310**

- `BillE` = $540 + (25,000 − 20,000) × $0.027 = $540 + $135 = **$675.00** (effective $0.027 a page). Pay-as-you-go would be 25,000 × $0.030 = $750.00, so the tier saves $75.00 this month, shared in proportion.
- Infra $100.99 = 10,099 cents. Exact shares: 2,423.76 · 1,615.84 · 2,019.80 · 1,211.88 · 1,615.84 · 1,211.88 → whole cents sum to 10,094 → 5 leftover cents go to the five largest fractions (.88, .88, .84, .84, .80).

17.3 Why the rounding rule matters

> Rounding each consumer's infra share to the nearest cent separately gives 24.24 + 16.16 + 20.20 + 12.12 + 16.16 + 12.12 = **$101.00** against a bill of **$100.99**. Over many months and consumers these cents break the "bill recovered exactly" promise. v2 allocates whole cents first and gives the leftover cents to the largest fractions (largest remainder), then asserts that the allocated amounts plus the Unallocated line equal each bill before the month is closed.

17.4 The commitment tier — risk and the month-to-date estimate

Month volume	Pay-as-you-go	20,000-page tier	Effective $/page (tier)	Better choice
15,000 pages	$450.00	$540.00	$0.0360	pay-as-you-go
18,000 pages	$540.00	$540.00	$0.0300	break-even
25,000 pages	$750.00	$675.00	$0.0270	tier

[B1] multiplies ledger pages by the pay-as-you-go price. v1 called it an upper bound; with the tier active it is not (at 15,000 pages it shows $450 while the bill is $540). v2 labels it a **PAYG-equivalent estimate** and forecasts the tier separately: fixed fee + overage above 20,000 pages. Buy the tier only after two or three months above ~18,000 pages.

18. Month end, reconciliation gate and capacity cost

18.1 Month-end procedure (operator, about 15 minutes)

1. A few days into the next month, take from Cost Management (`rg-docai-prod`): extraction cost and quantity, classification cost and quantity, everything else; convert each meter's unit to pages. Take the dev DI cost from `rg-docai-dev`.
2. [B2] Insert the `BillingPeriods` row. Any later correction of its numbers clears `ReconciledAtUtc`.
3. [B3] Reconcile **both meters**. A meter passes when the ledger is within ±2 % of Azure; zero on both sides passes; ledger pages with zero Azure pages always fail. [B3] also lists the month's `Unknown` Azure operations (calls that may have been billed without a result). Passing sets `ReconciledAtUtc` — **the hard gate**.
4. [B4] Close the month: refused unless reconciled; allocation in cents with largest remainder; costs of a meter that no consumer used go to `UnallocatedUsd`; a closed month cannot be closed twice.
5. [B5] / [B6] Publish the statements: per consumer, subtotal per company, per product, grand total.
6. Chargeback (later): a price plan on top of the cost; cost and price stay separate numbers.

[B7] corrections: a negative row that reverses one ledger row; a unique index allows only one reversal per row; the reversal is booked in the current open month.

18.2 Capacity and load-test cost (corrected volume math)

Quantity	v1	v2	How
Growth volume	30,000 pages/month	30,000 pages/month	unchanged
Pages per working day	≈ 1,000	≈ 1,364	30,000 ÷ 22 working days
Documents per day (3-page B/L)	≈ 350	≈ 455	1,364 ÷ 3
Busy hour (20 % of the day)	≈ 70	≈ 91 documents	about 1.5 a minute
Worker throughput (4 slots, 5–20 s per document)	12–48 / min	12–48 / min	8–32 × the busy hour (v1 said "more than ten times")
Load test: 10 × busy hour for 30 minutes	1,050 pages, $31.50	≈ 1,365 pages, ≈ $40.95	910 documents/hour × 0.5 h × 3 pages × $0.030

The conclusion holds: the limit is the time Azure takes per document, not throughput. The queue matters for bursts (a user uploading 50 B/Ls at once), and the takeover budget keeps one bad document from blocking a slot.

18.3 Cost controls

- **Idempotency:** the same file, for the same consumer, document type and requested line, is analyzed once. A new analysis needs a Failed job or an explicit `reprocess`.
- **Per consumer:** `MaxPagesPerJob` (10) and `DailyPageCap` (2,000), now counted over every run created today.
- **Azure budget alerts** on `rg-docai-prod` at 80 % and 100 %.
- **Billing follows Azure work:** no succeeded operation without its ledger row; no ledger row without a succeeded operation; a missing price stops the worker instead of dropping the cost.

19. Security and identity changes

Area	v2 rule	Finding
Pilot upload	`[Authorize]` on `DocsInController.UploadFile`; `Submit(IPrincipal …)` throws for an anonymous user. Operation-level authorization and CSRF for that action are recorded as go-live preconditions in the code-observations document.	F10
Webhook body	`<location path="api/DocAi/Webhook">`: `maxAllowedContentLength = 1048576` and `maxRequestLength = 1024` — the site allows 4 GB today. Signature headers checked before the body is read.	F16
Webhook sender	`AllowAutoRedirect = false`; default trust chain + host-name check; an optional, per-consumer private CA — never a blanket bypass.	F33, F38
Secrets	Key Vault secret cache 1 h; one rate-limited refresh after a 401 from the consumer.	F41
SQL	Bootstrap by an Entra admin; `docai_runtime` with INSERT-only ledger; operator and model-team roles.	F09, F26
App Service	`alwaysOn: true`; platform health check on `/health/live` (a dependency outage must not restart the only B1 instance); `/health/ready` feeds alerts.	F12, F37
Runtime	.NET 10 LTS (support to November 2028).	F14
Network	IP allow-list = defence in depth; runbook before tier changes; optional NAT Gateway. A network firewall cannot filter by URL path: allow the IPs to port 443 and add IIS `ipSecurity` for the webhook path (needs the section unlocked); real path-only exposure needs an L7 proxy.	F34, F39
Personal data	`Jobs.FileName` cleared at retention; company-side [C8] purges results; logs never hold values, file names, tokens or keys.	F29, F44
Input hygiene	`X-Correlation-Id` ≤ 64 chars of `[A-Za-z0-9-]`, else replaced; `IsFresh` compares Unix seconds without building a date (an extreme timestamp can no longer throw).	F51, F48

Unchanged and confirmed

- Managed identity everywhere, `disableLocalAuth` on Document Intelligence with a custom subdomain, shared keys off on Storage, Entra-only SQL, Key Vault RBAC, Azure Policy in Audit then Deny.
- Hashed 256-bit API keys with two rotation slots; consumer and tenant from the key, never from the body; registered callback URLs; 404 for another consumer's job.
- The identity verification checklist before go-live: a resource key gets 401, the managed identity gets 202, an account key fails, a SQL password login fails, a removed role fails the readiness probe.

20. Exceptions, error codes and alerts

Three kinds of error (unchanged), with v2 additions

Kind	Examples	Behaviour
Retryable	429, 5xx, timeouts, a lost lease, deadlock 1205	Back-off with Retry-After as a minimum; 5 attempts in the worker, 10 sends from the kit. A lost lease uses no attempt: the worker simply stops.
Configuration	401/403 from Azure, a missing role, Key Vault unreachable, **no price in force**, **model registry drift**	No attempt used; breaker pauses the worker 5 min; Critical alert.
Fatal	bad file, code not in the catalog, Azure 400, analysis failed, **unclassifiable document**, **crash loop**	Failed at once with an error code and a `job.failed` event for the run.

New and changed error codes

Code	Raised at	Meaning	Consumer action
`UNCLASSIFIED` new	worker	The classifier's class has no mapped line, or its confidence is under the threshold	submit again with the shipping line chosen
`WORKER_CRASH_LOOP` new	worker	The job was taken over 3 times (the document crashes the worker)	report it; the operator investigates
`ANALYSIS_RESULT_MISSING` new	worker	Azure answered 404 for a result that should still exist	submit again later; the operator is alerted
`SUBMIT_EXHAUSTED` new	kit	Ten sends failed (network, 5xx, 429)	submit again; check connectivity
`MODEL_NOT_FOUND` changed	worker	No active model — or Azure answered 404 for the registered model (registry drift alert)	submit again later
`RETRIES_EXHAUSTED` changed	worker	Five retryable errors, or a second Unknown submission in one run	submit again later
`PRICE_MISSING` new	worker, internal	No `AzureMeterPrices` row in force; jobs wait	nothing; the operator is alerted

Alerts

Alert	Condition	Severity	First action
Queue stuck	oldest Queued job over 5 min	2	worker health and the breaker
Breaker open	any 401/403 from Azure, or `PRICE_MISSING`	1	role assignments, local auth, price rows
Failure rate	over 5 % of an hour's jobs Failed	2	group by `ErrorCode`
Dead letter	any delivery DeadLettered	3	contact the consumer; replay [S12]
Readiness	`/health/ready` failing 5 min	1	the dependency the probe names
Budget	80 % / 100 % of the monthly budget	3 / 2	[B1] per consumer
Registry drift new	POST analyze answered 404	2	ModelRegistry vs the models in Azure
Crash loop new	any `WORKER_CRASH_LOOP`; any takeover above 1 per hour	2	memory, the document, the logs of that job
Unknown submissions new	any `Unknown` operation in a day	3	check for a double charge at month end

21. The MVC 4.8 kit



Figure 18. Submitting

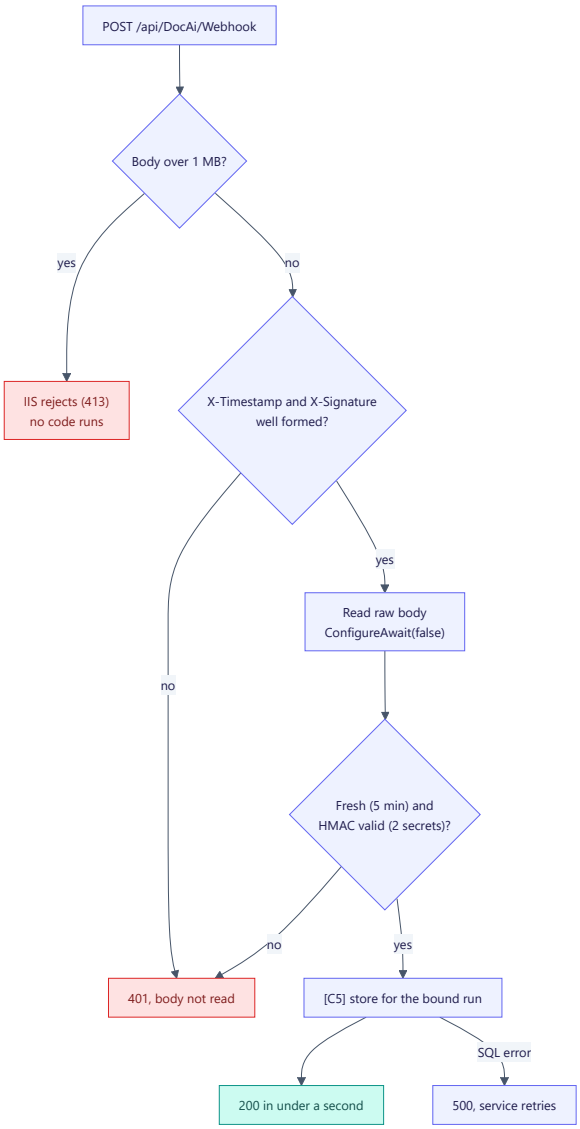


Figure 19. Receiving — cheap checks first

What a solution adds, and what it leaves alone

Adds	Leaves alone
4 files in a `DocAi` folder of the existing web project	no new project, no new assembly to deploy
1 route, where routes are mapped by hand (pilot: `WebApiConfig.cs`)	login, membership, cookies, roles, global filters, OWIN
1 table, `AiDocuments`	every existing table and stored procedure
2 settings, a secrets file, and a `<location>` limit for the webhook path	`packages.config` — HttpClient, JavaScriptSerializer and HMACSHA256 are in .NET Framework 4.8
Pilot only: `[Authorize]` on the one upload action; one Hangfire enqueue and one recurring job	`httpRuntime targetFramework` is **not** added (that would change the whole site's behaviour)

Code rules for the kit (legacy ASP.NET)

- The pilot's `<httpRuntime>` has no `targetFramework`, so ASP.NET runs its legacy synchronization context. Kit I/O uses `ConfigureAwait(false)` and reads every request value (`HttpContext.Current`, headers) before the first await.
- TLS: add TLS 1.2 only when the process is not on `SystemDefault` — `|=` on `SystemDefault` (0) would pin the whole process to TLS 1.2 and stop OS negotiation of newer protocols. The setting is process-wide.
- One static `HttpClient`; `ConnectionLeaseTimeout` of 5 min on the DocAi endpoint so DNS changes are picked up; per-call `CancellationTokenSource` with a 120 s transfer budget (20 MB over a 5 Mbps uplink takes at least 32 s — v1's 30 s timeout would fail every time).

- The startup check compares the tenant part of `consumerCode` from the catalog with `Defaults.UnEditableCompanyName` and stays off on a mismatch; it retries lazily instead of blocking application start.

Per-solution decisions — the pilot

Decision	ForwardingUniFreight
Where `Submit` is called	`DocsInController.UploadFile`, after each file's `SaveAs`
What the document belongs to	`'Operation'`, `Operations.ID`
How the file is sent	`BackgroundJob.Enqueue(() => DocAiClient.SendPending(localId))`
Recurring job	every 5 min: resend due Pending rows, refresh Submitted rows, [C7] file clean-up, [C8] retention
Webhook or polling only	webhook if the server accepts inbound HTTPS from the App Service outbound IPs; otherwise polling only
Controller type	Web API 5.2.3 `ApiController` (no session lock)

22. Coverage of the five concerns (recomputed)

Same rule as v1: an item is **covered** when it has a design, a place in the plan and a check that proves it; **partial** counts half; **later** counts zero. v1's percentages were calculated correctly, but several items it marked "covered" were not (the ledger missed billable work, the lease did not fence, the pilot action was not authorized). v2 rebuilds the checklists.

#	Concern	v1	v2	v2 items	Still open
1	Cost per application on a shared resource	85 %	**87 %**	13 covered, 0 partial, 2 later of 15 · new: ledger per operation, missing price stops, both meters reconciled as a gate, cent rounding, one reversal per row	Cost Management import (later) · chargeback (later)
2	Performance	88 %	**91 %**	13 covered, 3 partial of 16 · new: Always On, separate analyze/poll limits, appending clustered key, pilot sends off the request	targets measured in the pilot · load test · scale-out path
3	Security	88 %	**89 %**	19 covered, 3 partial, 1 later of 23 · new: least-privilege SQL roles, webhook body cap, no redirects + trusted TLS, upload guard, admin bootstrap	webhook IP restriction · pen test · operation-level auth + CSRF on the upload (go-live precondition) · private networking (later)
4	Exception handling and logging	93 %	**95 %**	18 covered, 2 partial of 20 · new: lease fencing, run versioning, poison budget, Unknown submissions, kit status classification, liveness vs readiness	runbook per alert · each solution's own logging
5	No core authorization change, no new project	100 %	**100 %**	13 covered of 13 · new: webhook limit in one `<location>` element; same file on two records keeps both links	—

Arithmetic: (13 + 0) ÷ 15 = 86.7 % · (13 + 1.5) ÷ 16 = 90.6 % · (19 + 1.5) ÷ 23 = 89.1 % · (18 + 1) ÷ 20 = 95.0 % · 13 ÷ 13 = 100 %. Concern 5 stays at 100 % because the pilot's `[Authorize]` is one attribute on one action — login, membership, global filters and OWIN are untouched.

Appendix A — findings register

Found by: C = Claude, X = Codex, CX = both. Severity is the value agreed after the debate. "v1 location" refers to the sections and statement labels of the source plan.

ID	Sev.	By	v1 location	Problem	v2 fix
F01	High	CX	§7 [S3]–[S9]	No lease ownership: a worker that lost its lease still writes; "0 rows = 5th failure" is ambiguous	`LeaseToken` on every write; exhaustion decided in code
F02	High	CX	[S2] [S9] [C5]	A late failure event of an earlier attempt defeats the requeued job; the result is lost	`RunNo`; exact run binding; Superseded events
F03	High	XC	[S9]–[S12]	An old sender can acknowledge a re-armed event; a late failure can revert Delivered to Pending; 10 × 15 s exceeds the 2-min lease	Immutable events; token + Pending guard on [S11]; 5-min lease; parallel sends
F04	High	CX	Fig. 7, Jobs, [S4] [S8]	Classify-then-extract shares one Operation-Location; classifier facts lost on a crash	`AzureOperations`; `Stage`; [S18] [S19]
F05	High	X	§6, [S4], §15 test 3	"Crash right after POST → no second POST" cannot be guaranteed	Sending row before the POST; Unknown state; claim reworded
F06	High	CX	[S8] [S9], `UX_Usage_Job_Meter`	Billable Azure work on failed jobs never reaches the ledger	Ledger row per succeeded operation
F07	High	CX	[S8]	`CROSS APPLY` on prices silently drops a meter when no price is in force	THROW; breaker; alert
F08	High	CX	[S3] [S9]	A document that crashes the worker, or a failure that cannot be persisted, loops forever	`TakeoverCount` ≥ 3 → `WORKER_CRASH_LOOP`; bounded payloads
F09	High	CX	§7 migration, §11	`db_ddladmin` cannot create users or grant roles; name hard-coded per environment	Admin bootstrap with `WITH OBJECT_ID`; grants script
F10	High	X	§13 pilot table	`DocsInController.UploadFile` is anonymous	`[Authorize]` + kit guard
F11	High	X	[C1], Submit	The same file on a second operation returns the first operation's row	Dedupe by line + entity
F12	High	CX	§1, §4, §15	No Always On: the hosted worker stops when the app is idle	`alwaysOn: true` + idle test
F13	High	C	[S2], idempotency index	A job completed with the wrong user-picked line can never be reprocessed	Requested line in the key; `reprocess=true`
F14	High	XC	§1, §4	.NET 9 support ends 10 Nov 2026	.NET 10 LTS
F15	Med.	X	PayloadJson, ErrorMessage	A 1,000-char message can exceed `nvarchar(2000)` JSON; consumer column was 500	400-char messages; `nvarchar(max)` payload
F16	Med.	X	§13 webhook	The body is read in full before the 1 MB check; IIS allows 4 GB	`<location>` limits; header pre-check
F17	Med.	CX	[C3]	A late 202 resets a fast `job.failed` back to Submitted	[C3] only from Pending
F18	Med.	X	[C6]	Select + unconditional update: duplicates, and views can postpone refresh forever	Atomic `UPDATE … OUTPUT`
F19	Med.	X	[C1] [C4] [C6]	After 10 failed sends the row stays Pending forever and blocks a new upload	`SUBMIT_EXHAUSTED`; retry = new row
F20	Med.	CX	[C4], DocAiClient	Every 4xx is permanent, including 429 and 401	Status classification; Retry-After
F21	Med.	X	[S1] [S2]	Requeues are not counted in the daily cap; the cap runs before the duplicate lookup	Cap over `JobRuns`; lookup first
F22	Med.	X	[B3]	Classification never reconciled; zero Azure pages gives NULL	Both meters; zero rules; gate

F23	Med.	CX	[B4] [B5]	Cent rounding and zero-use months break full cost recovery	Largest remainder; Unallocated line
F24	Med.	X	[B7]	Two reversals of one row can both pass `NOT EXISTS`	Unique reversal index
F25	Med.	X	[B1]	PAYG is not an upper bound under the tier	Relabelled; separate tier forecast
F26	Med.	CX	§7, §11	`db_datawriter` can rewrite the ledger, keys and models	Custom roles
F27	Med.	X	[S14]	Activation can retire one line's model and activate another line's	Group derived from the new model
F28	Med.	CX	§7 decisions	UUIDv7 does not append in a SQL Server clustered index	`JobSeq` clustered
F29	Med.	X	[C7], §10	The company-side result retention is advertised but not implemented	[C8]
F30	Med.	XC	§6 retry table	A GET 404 is not proof of expiry; API version not pinned	24 h rule from acceptance; `2024-11-30`
F31	Med.	XC	Fig. 7, §9	Classifier mapping, threshold and split mode undefined; polling cap below Retry-After	`ClassifierClasses`; `splitMode none`; token buckets
F32	Med.	XC	§13 HttpClient	Static client never refreshes DNS	`ConnectionLeaseTimeout`
F33	Med.	XC	§13, §16	Following a redirect to a 200 login page marks a delivery Delivered	`AllowAutoRedirect = false`
F34	Med.	CX	§13 step 6, §16	Outbound IPs are shared and change with the tier	Defence in depth; runbook; optional NAT
F35	Med.	C	§13, §9	Pilot upload is a sync multi-file request; inline sends stack 30 s timeouts	Hangfire enqueue + reconciler
F36	Med.	C	§13 DocAiClient	30 s timeout vs a 20 MB file on a slow uplink	120 s transfer budget
F37	Med.	C	§11 readiness	A platform probe on readiness can restart the only instance during an Azure outage	Liveness for the platform
F38	Med.	C	§16	Internal-CA certificates on consumer IIS are rejected by the sender	Trusted chain; optional per-consumer CA
F39	Med.	C	§13 step 6	A network firewall cannot filter by URL path	IPs to :443 + IIS path rule; L7 for path-only
F40	Med.	C	§6 retry table	POST analyze 404 is unmapped	`MODEL_NOT_FOUND` + drift alert
F41	Med.	C	§5 step 6	No refresh policy for the cached webhook secret	1 h TTL + refresh after 401
F42	Med.	C	§13 sketches	Legacy ASP.NET synchronization context in the pilot	`ConfigureAwait(false)`; no global retarget
F43	Med.	C	[S2]	Serializable range locks can deadlock (1205)	Retry the whole transaction
F44	Med.	C	[S15]	`Jobs.FileName` (personal data) kept forever after soft delete	Cleared at retention
F45	Med.	C	[C3], §13	A 200 for an already Completed job waits 10 min for a refresh	Immediate GET
F46	Med.	CX	[C7], §13	Housekeeping needs a scheduler the plan says is not required	Hangfire where present; best-effort otherwise
F47	Low	XC	§13 TLS line	`|= Tls12` on `SystemDefault` pins the process to TLS 1.2	Guard
F48	Low	X	§13 IsFresh	An extreme timestamp throws before signature rejection	Unix-seconds comparison
F49	Low	C	§11 roles	The split Worker needs only Blob Data Reader (results are in SQL)	Role table corrected

F50	Low	C	§8, §9	Growth infra leaves out storage and Key Vault; volume per working day understated	$100.99; 1,364 pages a working day
F51	Low	C	§12 correlation	An untrusted correlation header goes straight into logs and SQL	Length and charset check
F52	Low	C	IX_Jobs_Queue, BlobPath	`IsDeleted` outside the queue index; every blob named `.pdf`	Index filter; real extension
F53	Low	X	[S3] [S10]	READPAST under Azure SQL's default RCSI was never tested (High withdrawn in the debate)	Integration test with RCSI ON

Appendix B — debate outcomes and facts

Topic	Positions	Resolution
READPAST under RCSI	Codex: the claims fail on Azure SQL (High). Claude: Hangfire uses the same hints on Azure SQL in production.	Codex withdrew the High. Keep `ROWLOCK, UPDLOCK, READPAST`. Codex also showed that Claude's idea of adding `READCOMMITTEDLOCK` would conflict with `ROWLOCK` (same hint group). Add an RCSI integration test.
TLS line in the kit	Codex: Medium. Claude: the pilot runs in 4.0 compatibility, so the line is needed there.	Low; guard against `SystemDefault`; document that the setting is process-wide.
Error-size mismatch	Codex: High. Claude: the service writes its own messages.	Medium for the size; the poison loop it can cause is its own High (F08).
.NET 9	Codex: Medium. Claude: raise it.	High; .NET 10 LTS from the start.
`IsFresh` overflow	Codex: Medium. Claude: an unsigned caller only gets a 500.	Low; the fix is one line.
Run binding rule	Claude: accept `runNo >=` the stored one. Codex: that can attach an unrequested run.	Exact `(RemoteJobId, RemoteRunNo)`; RunNo grows only for a new authorized analysis.
Lease guard	Codex: add `LeaseExpiresAtUtc > now`.	Token only: it changes on every claim, so it fences; an expiry check would make a sole slow owner give up for nothing.
Housekeeping	Claude: piggyback on Refresh. Codex: best-effort only.	Hangfire where present; best-effort otherwise, stated as such.
Webhook path exposure	Claude: IIS path `ipSecurity`. Codex: it limits that path, not the others.	Firewall to :443 + IIS path rule; true path-only exposure needs an L7 proxy or a dedicated binding.

Facts confirmed in this review

- Delete Analyze Result exists in Document Intelligence API `2024-11-30`; results are kept 24 hours from operation completion.
- S0 defaults: 15 analyze and 50 GET requests per second; ten free custom-neural training hours a month.
- Always On is available on Basic and is needed for continuous background work; apps can unload after about 20 idle minutes.
- App Service outbound IPs come from a shared pool per deployment unit and can change with the pricing tier; a NAT Gateway gives a stable address.
- `CREATE USER … FROM EXTERNAL PROVIDER` supports `WITH OBJECT_ID`.
- UUIDv7 does not reduce fragmentation as a SQL Server `uniqueidentifier` clustered key without byte reordering.
- .NET 9 support ends 10 November 2026; .NET 10 is the current LTS.

Still unverified (check before go-live)

- The France Central prices of 11 October 2026 ($30 / $3 per 1,000 pages) and the commitment-tier quote ($540 for 20,000 pages, $27 per 1,000 above).
- The infrastructure prices and the NAT Gateway price (Azure pricing calculator).
- Whether Azure bills an analysis that ends in `failed` (the ledger records succeeded operations only; reconciliation shows any difference).

Appendix C — next steps

Nothing has been implemented. After this explanation is approved, the next step is to write the revised plan document, `AI-Extraction-Service-Plan-v2.md`, with every fix applied in place, and to verify it.

The v2 document will contain

- §1 / §4: .NET 10, Always On, the new tables and components.
- §5 / §6: flow, the three request-cycle diagrams, the worker flowchart, the retry rules, the state machines, the API contract with `runNo` and `reprocess`, and two HMAC test vectors (below).
- §7: full DDL, the bootstrap and grants scripts, statements [S1]–[S19] and [C1]–[C8].
- §8: billing statements [B1]–[B7] with the reconciliation gate and largest-remainder rounding; the corrected examples.
- §9–§16: performance, security, identity, exceptions, the kit, rules, deliverables and notes, as in sections 18–21 of this document.
- Appendix: the findings register and the debate outcomes.

HMAC test vectors

Vector	Body	X-Timestamp	Expected X-Signature
1 — algorithm (from v1, unchanged)	the v1 `job.failed` line, 326 bytes	`1793268922`	`sha256=4dad37deda733dbf91e6907cd9b718ef45ae55cd6b44a4f382e32258f3d31e14`
2 — v2 envelope	`job.failed` with `"runNo":2` and `UNCLASSIFIED`, 346 bytes	`1793271763`	`sha256=4489eab28971b0bd17ccc6b66e0e2eba5b12b33c7ddc9d690686afbad068b168`

Secret for both (test value only, never a real secret): `ZG9jYWktdGVzdC12ZWN0b3Itc2VjcmV0LTMyYnl0ZXM=`. Vector 2 body:

```
{"eventId":"9c4f2a1e-6b3d-4e8a-b7c5-1d2e3f4a5b6c","eventType":"job.failed","jobId":"0192f0a4-7c1e-7b3a-9d2e-5f6a7b8c9d01","runNo":2,"clientRequestId":"3a8e5c71-2b4d-4f9e-8c1a-6d7e8f9a0b12","status":"Failed","error":{"code":"UNCLASSIFIED","message":"Classifier confidence 0.41 is under 0.700"},"result":null,"occurredAtUtc":"2026-10-29T11:02:41Z"}
```

How the v2 document will be verified

1. Recompute both HMAC vectors.
2. Recompute both cost examples, the rounding rule and every coverage percentage by script.
3. Run the DDL and statements on a SQL Server 2022 container with `READ_COMMITTED_SNAPSHOT ON`: the scripts parse, constraints accept every statement, an update after a lost lease changes 0 rows, [S8] throws on a missing price, [B4] adds up to the bill.
4. Search the document for leftover v1 claims (".NET 9", "upper bound", "no second POST", `db_datawriter`).
5. Tests listed in the deliverables:
   - stale `job.failed` against run 2, and a webhook that arrives before the submit reply;
   - lease loss mid-poll, a crash between Sending and Accepted, a classifier crash, three takeovers;
   - the claim under RCSI, 429 on submit, the same file on two operations;
   - an unsigned 2 MB webhook, and the sender ignoring a 302.
6. A final consistency pass by Codex.